



Universidad Católica
de Santiago del Estero
Departamento Académico Rafaela

**Scouting Inteligente con Inteligencia Artificial aplicado al Fútbol
Juvenil**

*Evaluación y predicción del potencial de desarrollo mediante análisis
de datos*

**Trabajo Final
Ingeniería en Informática**

Alumno: Solari, Pablo.

Prof: Duarte, Jorge – Lovera, Maximiliano

Año: 2026.

ÍNDICE

LISTA DE FIGURAS.....	5
LISTA DE TABLAS.....	6
RESUMEN.....	7
1. INTRODUCCIÓN.....	1
1.1 Descripción del problema.....	1
1.2 Objetivos.....	3
1.3 Antecedentes.....	4
1.3.1 Influencia del Big Data y la IA en el fútbol.....	5
1.3.2 Importancia del software en el fútbol formativo.....	5
1.4 Justificación.....	6
1.5 Alcance.....	6
2. MARCO TEÓRICO.....	9
2.1 Definición de términos.....	9
2.1.1 Aprendizaje automático aplicado al deporte.....	9
2.1.2 Análisis de datos en el deporte.....	10
2.1.3 Relevancia de la inteligencia artificial en el deporte.....	11
2.1.4 Ciencia de datos y Big Data.....	12
2.1.5 Analítica de datos.....	13
2.1.6 Tecnologías consideradas y utilizadas.....	14
2.1.7 Fundamentos de modelado y evaluación.....	15
2.2 Trabajos relacionados y contexto de uso.....	15
3. METODOLOGÍA.....	17
3.1 Enfoque de datos y modelado.....	17
3.1.1 Recolección y estructura de datos.....	17
3.1.2 Preprocesamiento y normalización.....	17
3.1.3 Entrenamiento y evaluación.....	17
3.1.4 Definición de la variable objetivo.....	17
3.2 Metodología de desarrollo.....	18
3.3 Ciclo de vida del software.....	18
3.4 Desglose de tareas y cronograma consolidado.....	19

4. ANÁLISIS Y DISEÑO.....	20
4.1 Arquitectura del sistema.....	20
4.1.1 Arquitectura general, capas y componentes.....	20
4.1.2 Diseño de la Base de datos.....	31
4.1.3 Interfaz de Usuario.....	33
4.2 Funcionalidades y componentes.....	33
4.2.1 Módulos funcionales implementados.....	33
4.2.2 Modelo de evaluación deportiva.....	34
4.2.3 Uso de PyTorch y Scikit-learn.....	34
4.2.4 Seguridad y autenticación.....	35
4.3 Requisitos del sistema.....	36
4.4 Modelo de datos.....	36
4.5 Endpoints principales del MVP.....	38
4.6 Decisiones tecnológicas y alineación con el MVP.....	39
4.7 Casos de uso.....	39
5. DESARROLLO.....	24
5.1 Implementación del backend.....	43
5.2 Implementación del frontend.....	44
5.3 Datos, edad y categoría juvenil.....	45
5.4 Entrenamiento del modelo de aprendizaje automático.....	45
5.5 Seguridad y robustez implementada.....	46
5.6 Despliegue del sistema.....	47
5.7 Problemas técnicos y decisiones de implementación.....	49
6. PRUEBAS Y RESULTADOS.....	50
6.1 Plan de pruebas.....	50
6.2 Resultados del MVP.....	50
6.2.1 Métricas de PlayerNet y calibración.....	51
6.2.2 Comparación con baseline.....	51
6.2.3 Evidencia operativa del deploy.....	52
6.2.4 Evidencia de integración continua.....	61
6.3 Limitaciones observadas en la validación.....	62

7. CONCLUSIÓN.....	64
7.1 Conclusiones.....	64
7.2 Cumplimiento de objetivos.....	64
7.3 Limitaciones.....	65
7.4 Lecciones aprendidas.....	65
7.5 Trabajo futuro.....	65
8. BIBLIOGRAFÍA.....	67
9. ANEXOS TÉCNICOS DEL MVP.....	69
9.1 Rutas principales.....	69
9.2 Variables de entorno relevantes.....	69
9.3 Glosario técnico.....	69
9.4 Reproducción mínima.....	72
10. ANEXOS VISUALES Y EVIDENCIA.....	73
10.1 Evidencia Render.....	73
10.2 Integración continua y documentos técnicos.....	73
10.3 Limitaciones documentadas del MVP.....	74
10.4 Diagramas ampliados.....	76
10.5 Casos de uso ampliados.....	81

LISTA DE FIGURAS

Figura 4-1. Diagrama de componentes de TPScouting.....	21
Figura 4-2. Diagrama de clases del modelo de dominio.....	37
Figura 4-3. Diagrama de casos de uso general.....	40
Figura 4-4. Diagrama de casos de uso de gestión de jugadores.....	40
Figura 4-5. Diagrama de casos de uso de análisis y decisión scout.....	41
Figura 5-1. Secuencia de predicción de potencial de jugador.....	43
Figura 5-2. Secuencia de carga del panel general.....	44
Figura 5-3. Diagrama de despliegue en Render.....	47
Figura 6-1. Curva real de entrenamiento generada desde training_metadata.json.....	52
Figura 6-2. Login del MVP desplegado en Render.....	53
Figura 6-3. Panel general o mesa de scouting con métricas accionables.....	54
Figura 6-4. Listado paginado de jugadores con edad, categoría y potencial.....	55
Figura 6-5. Ficha de jugador con historial, atributos y acciones CRUD en modales.....	56
Figura 6-6. Vista de predicción de potencial del jugador.....	59
Figura 6-7. Comparador múltiple de jugadores.....	60
Figura 6-8. Resumen público de la ejecución CI #71 en GitHub Actions.....	62
Figura 10-1. Diagrama de componentes de TPScouting ampliado.....	76
Figura 10-2. Diagrama de clases del modelo de dominio ampliado.....	77
Figura 10-3. Diagrama de secuencia de predicción ampliado.....	78
Figura 10-4. Diagrama de secuencia del panel general ampliado.....	79
Figura 10-5. Diagrama de despliegue en Render ampliado.....	80
Figura 10-6. Diagrama de casos de uso general ampliado.....	81
Figura 10-7. Diagrama de casos de uso de gestión de jugadores ampliado.....	82
Figura 10-8. Diagrama de casos de uso de análisis y decisión scout ampliado.....	83
Figura 10-9. Historial público y ejecución exitosa del workflow CI de TPScouting.....	73

LISTA DE TABLAS

Tabla 3-1. Desglose de tareas y cronograma consolidado.....	19
Tabla 4-1. Comparación entre Flask y .NET Core Minimal APIs.....	27
Tabla 4-2. Comparación entre SQLAlchemy y Peewee.....	29
Tabla 4-3. Comparación entre motores relacionales.....	32
Tabla 4-4. Capas y responsabilidades del MVP.....	31
Tabla 4-5. Dimensiones de evaluación deportiva.....	34
Tabla 4-6. Requisitos funcionales del sistema.....	36
Tabla 4-7. Entidades principales del modelo de datos.....	37
Tabla 4-8. Endpoints principales del MVP.....	38
Tabla 4-9. Decisiones tecnológicas.....	39
Tabla 4-10. Casos de uso principales.....	39
Tabla 5-1. Configuración verificada del modelo PlayerNet.....	46
Tabla 5-2. Estado del despliegue en Render.....	47
Tabla 6-1. Cobertura del plan de pruebas.....	50
Tabla 6-2. Resultados de validación automatizada y CI.....	51
Tabla 6-3. Métricas de PlayerNet calibrado y crudo en test.....	51
Tabla 6-4. Comparación con baselines.....	52
Tabla 6-5. Smoke real del despliegue en Render.....	52
Tabla 7-1. Cumplimiento de objetivos.....	65
Tabla 9-1. Rutas principales.....	69
Tabla 9-2. Variables de entorno relevantes.....	69
Tabla 9-3. Glosario técnico.....	71

RESUMEN

Este Trabajo Final presenta TPScouting, un producto mínimo viable (MVP) académico orientado al seguimiento de jugadores juveniles de fútbol. La aplicación permite registrar jugadores, atributos, estadísticas, reportes y disponibilidad; visualizar información en un panel general; comparar perfiles; y estimar una proyección de potencial mediante PlayerNet, un modelo implementado en PyTorch. La validación utiliza datos sintéticos, pruebas automatizadas, métricas de clasificación, comparación con modelos de referencia y evidencia del despliegue. Los resultados demuestran la factibilidad técnica de integrar el flujo completo, pero no constituyen una validación deportiva real ni prueban que el sistema reemplace el criterio de un scout.

Palabras clave: scouting juvenil; fútbol; aprendizaje automático; Flask; PyTorch.

1. INTRODUCCIÓN

1.1 Descripción del problema

En el dinámico mundo del fútbol contemporáneo, los clubes de comunidades pequeñas y de recursos limitados enfrentan desafíos significativos, especialmente en el ámbito del fútbol infantojuvenil. En muchos casos, estas instituciones constituyen el primer espacio de formación deportiva y social para niños y adolescentes, pero se ven condicionadas por restricciones financieras, logísticas y de infraestructura que dificultan el seguimiento sistemático de sus jóvenes jugadores.

Esta situación plantea una paradoja: mientras el talento juvenil puede convertirse en una fuente de valor deportivo y económico en el mediano plazo, la falta de inversión y de procesos de registro y análisis de información reduce la capacidad de los clubes para identificar oportunidades, planificar la formación y sostener decisiones basadas en evidencia.

Además, la recolección y el análisis de datos suelen ser insuficientes. Aunque se generan registros a través de entrenamientos, partidos y evaluaciones, con frecuencia dichos datos no se almacenan de forma estructurada o se pierden, lo que limita la posibilidad de comparar jugadores, detectar patrones de evolución y evaluar el potencial con criterios consistentes.

En consecuencia, muchos jóvenes con potencial de desarrollo pueden pasar desapercibidos o no recibir seguimiento adecuado. Esto impacta tanto en sus oportunidades de crecimiento deportivo y profesional como

en la sostenibilidad de los clubes, que dependen de maximizar el rendimiento de recursos formativos reducidos.

Frente a esta realidad, el presente proyecto propone una solución tecnológica que integra ciencia de datos e inteligencia artificial para complementar el scouting tradicional. El objetivo es aportar mayor objetividad, trazabilidad y capacidad de comparación, mejorando la toma de decisiones y ampliando la visibilidad de talentos en contextos con recursos limitados.

Con la expansión de la IA y su acercamiento al ámbito cotidiano, esta dinámica está experimentando un cambio relevante en las formas de trabajo del scouting. Su capacidad para procesar y analizar grandes volúmenes de datos ofrece una nueva perspectiva para la identificación y el desarrollo de jóvenes talentos. Herramientas basadas en IA, como sistemas de análisis de rendimiento y algoritmos predictivos, brindan conclusiones más profundas y precisas sobre el potencial y las habilidades de los jugadores. Por ejemplo, plataformas como Wyscout y Opta Pro, utilizadas profesionalmente, permiten análisis detallados del rendimiento en campo y mejoran la toma de decisiones. También favorecen entrenamientos personalizados, adaptados a las necesidades y fortalezas individuales y colectivas. No obstante, la adopción de estas tecnologías enfrenta desafíos en clubes con recursos limitados, tanto por restricciones presupuestarias como por falta de formación técnica específica.

Este proyecto aborda el problema mediante una solución tecnológica basada en ciencia de datos e inteligencia artificial, integrada de manera accesible en el proceso de evaluación y seguimiento del fútbol juvenil.

Problema de investigación: ¿Cómo pueden la ciencia de datos y la inteligencia artificial contribuir a optimizar la identificación, la evaluación y la proyección del potencial de jugadores juveniles en clubes con recursos limitados?

Una de las principales limitaciones de los clubes de fútbol con recursos reducidos radica en optimizar sus inversiones y gestionar eficazmente la información de los jugadores. En este contexto, cada decisión debería contribuir al desarrollo institucional y al proceso formativo de los futbolistas juveniles.

Otra dificultad relevante es la recolección y el análisis de datos. Aunque los clubes generan información sobre el rendimiento y las habilidades de sus jugadores, con frecuencia carecen de medios para procesarla de manera sistemática. Los registros en papel o las herramientas rudimentarias pueden producir pérdida, duplicación o uso deficiente de información. Este enfoque manual no solo resulta ineficiente, sino que también puede conducir a decisiones mal fundamentadas sobre el desarrollo de los deportistas.

Asimismo, la dificultad para identificar patrones, tendencias y señales de evolución limita la toma de decisiones estratégicas. Esto puede ocasionar que algunos talentos no sean reconocidos a tiempo y que los recursos formativos se asignen sin criterios consistentes, afectando tanto a la sostenibilidad del club como a las oportunidades de desarrollo de los jugadores.

Por lo tanto, esta investigación busca dar respuesta a la pregunta planteada, con el objetivo de presentar una solución que permita a los clubes optimizar sus recursos y gestionar de mejor forma la identificación y el desarrollo de sus jóvenes promesas a través del uso efectivo de tecnologías de IA.

1.2 Objetivos

Objetivo general: desarrollar una aplicación que integre inteligencia artificial y análisis de datos para mejorar la evaluación y la proyección del potencial de desarrollo de futbolistas juveniles, utilizando información proveniente de entrenamientos, estadísticas de partidos y

evaluaciones periódicas, con el fin de ofrecer una base más objetiva y accionable para la toma de decisiones.

- Optimizar la evaluación: utilizar técnicas de analítica de datos y modelos de aprendizaje automático para procesar información deportiva y estimar habilidades con criterios consistentes, reduciendo la subjetividad del proceso tradicional.
- Identificar talentos subrepresentados: detectar perfiles con proyección que podrían pasar desapercibidos bajo enfoques convencionales, ampliando el espectro de detección.
- Mejorar la toma de decisiones: proporcionar a entrenadores y responsables del desarrollo información basada en datos para decisiones de formación, seguimiento y proyección.
- Promover la igualdad de oportunidades: democratizar el acceso a herramientas de scouting basadas en datos para acercar posibilidades de identificación y desarrollo a juveniles de clubes con menos recursos.
- Optimizar la gestión de recursos: apoyar, mediante evidencia, la asignación y priorización de recursos formativos limitados.

Criterios de éxito: en el marco de este MVP, el resultado se evalúa mediante criterios técnicos observables: funcionamiento de los flujos principales, ejecución de pruebas automatizadas, disponibilidad de evidencia operativa, métricas del modelo y documentación reproducible. El impacto real sobre la eficiencia, la efectividad, la igualdad de oportunidades o las decisiones deportivas requiere una validación posterior con clubes, scouts y jugadores reales.

1.3 Antecedentes

El software se inspira en la creciente tendencia del uso de datos e inteligencia artificial en el scouting deportivo. Los clubes de fútbol han

comenzado a utilizar herramientas avanzadas de análisis para identificar talento, reducir riesgos en fichajes y optimizar el rendimiento de sus equipos juveniles.

1.3.1 Influencia del Big Data y la IA en el fútbol

- El análisis de eventos, estadísticas e historiales permite construir evaluaciones multidimensionales y comparar perfiles de jugadores con criterios consistentes (Pappalardo et al., 2019; Decroos et al., 2019).
- Plataformas profesionales como Wyscout centralizan video y datos deportivos para apoyar procesos de observación, comparación y reclutamiento (Hudl, s. f.).
- La literatura reciente también explora modelos de aprendizaje automático para detectar perfiles de alto potencial y pronosticar la evolución futura de los jugadores, aunque sus resultados dependen de la calidad de los datos, la definición del objetivo y el contexto de aplicación (Lacan, 2024; van Arem et al., 2025).

1.3.2 Importancia del software en el fútbol formativo

- Los clubes pequeños y medianos tienen menos acceso a tecnología avanzada, lo que genera ineficiencia en la captación de talento.
- Actualmente, los métodos de scouting son subjetivos y basados en observación, lo que puede llevar a errores en la identificación del potencial de un jugador.
- Este software busca reducir la brecha tecnológica, permitiendo que cualquier club o academia tenga acceso a herramientas basadas en datos para la toma de decisiones.

1.4 Justificación

La propuesta de este sistema representa la unión entre tecnología y deporte, con foco en el fútbol juvenil. Al incorporar registro estructurado, visualizaciones y un módulo predictivo, se busca mejorar la precisión del scouting y reducir brechas tecnológicas entre clubes de distinto poder económico.

- Innovación tecnológica: integración de analítica de datos e inteligencia artificial aplicada a scouting juvenil.
- Aplicación práctica: soporte a clubes para registrar, comparar y proyectar jugadores de manera operativa y accesible.
- Contribución a la igualdad: reducción de desigualdades al ofrecer una herramienta replicable para clubes con recursos limitados.
- Caso práctico: demostración de factibilidad mediante un MVP web integrado con un módulo de predicción.
- Visualización y comunicación de datos: Mediante la presentación gráfica y la generación de informes basados en los análisis realizados, el sistema facilita la interpretación de los datos y también, mejora la comunicación y comprensión de la información por parte de entrenadores, directivos y otras partes interesadas.

1.5 Alcance

El alcance del proyecto es el desarrollo de un MVP académico de scouting juvenil que permite registrar jugadores, cargar atributos y estadísticas, consultar fichas, comparar perfiles y calcular una proyección de potencial mediante un modelo de Machine Learning integrado al backend Flask.

La solución se enfoca en una institución o entorno de evaluación acotado, con datos sintéticos para demostrar el flujo completo sin depender de proveedores externos de datos deportivos.

Incluye:

- Autenticación por usuario y contraseña, roles básicos y protección de rutas sensibles.
- Gestión de jugadores, staff, atributos, rendimiento, disponibilidad, evaluaciones físicas, reportes y comparadores.
- Carga individual y carga masiva por CSV compatible con Excel, con previsualización y validación por fila.
- Dashboard o panel general con métricas accionables adaptadas al rol del usuario.
- Inferencia de potencial con PlayerNet en PyTorch, usando features derivadas de atributos, edad, posición e historial disponible.
- Pruebas automatizadas con pytest, cobertura en CI y smoke real sobre Render.

Queda fuera del alcance del MVP:

- Análisis automático de video, tracking GPS, integración con Wyscout/Hudl u otros proveedores comerciales.
- Gestión multi-club, módulos financieros, contratos, valuación económica o scouting internacional distribuido.
- Entrenamiento con datos reales longitudinales de clubes, ya que el MVP utiliza datos sintéticos para reproducibilidad académica.

- Arquitectura productiva multi-instancia con cache distribuido, rate limiting centralizado, auditoría completa y orquestación de pipelines.

Criterios mínimos de éxito observables:

- Ejecutar la suite automatizada sin fallos.
- Acceder al deploy público, iniciar sesión y navegar dashboard, listado, ficha, predicción y comparadores.
- Mantener atributos dentro de escala 1-20 y potencial categorizado en bajo, medio y alto con umbrales documentados.
- Registrar evidencia real de métricas de entrenamiento, smoke de Render y capturas de funcionamiento.

2. MARCO TEÓRICO

2.1 Definición de términos

2.1.1 Aprendizaje automático aplicado al deporte

Definición y Principios Básicos de Machine Learning

Machine Learning (ML) es una rama de la inteligencia artificial que permite a las máquinas aprender y mejorar a partir de la experiencia, sin ser explícitamente programadas para cada tarea. Esencialmente, ML se centra en el desarrollo de algoritmos que pueden procesar datos y realizar predicciones o tomar decisiones basadas en esos datos (Hastie et al., 2009; Goodfellow et al., 2016).

Subcampos del Machine Learning

Aprendizaje Supervisado: Esta técnica se basa en aprender a partir de un conjunto de datos etiquetados. Por ejemplo, en el deporte, puede usarse para predecir el resultado de un partido basándose en datos históricos.

Aprendizaje No Supervisado: Se utiliza para encontrar patrones ocultos o agrupaciones intrínsecas en datos no etiquetados. Por ejemplo, puede identificar agrupaciones naturales de jugadores basadas en su rendimiento.

Aprendizaje por Refuerzo: Implica aprender a tomar decisiones mediante la realización de ciertas acciones y observación de los resultados o recompensas. Por ejemplo, puede aplicarse para desarrollar estrategias de juego.

Aplicación en el Deporte: ML puede utilizarse para una variedad de aplicaciones en el deporte, incluyendo la optimización de entrenamientos, análisis táctico, y prevención de lesiones. Permite a

entrenadores y analistas deportivos obtener descubrimientos profundos y basados en datos que antes eran imposibles o muy difíciles de obtener. Se aplica para análisis de rendimiento, prevención de lesiones, personalización del entrenamiento, y scouting de jugadores. Incluye ejemplos específicos, como modelos predictivos para identificar futuros talentos o sistemas de análisis de movimiento para optimizar técnicas deportivas.

Cómo las técnicas de Machine Learning mejoran las predicciones y decisiones en el deporte

Predicciones Basadas en Datos: Los algoritmos de Machine Learning pueden analizar grandes volúmenes de datos deportivos para predecir resultados de partidos, rendimiento de jugadores, y tendencias de juego.

2.1.2 Análisis de datos en el deporte

Importancia del Análisis de Datos

Transformación del deporte moderno: El análisis de datos ha revolucionado el deporte moderno, convirtiéndose en un elemento esencial para la toma de decisiones estratégicas. Con la capacidad de recopilar y analizar grandes volúmenes de datos, los equipos pueden obtener insights profundos que antes eran inaccesibles.

Toma de decisiones basada en evidencia: Los datos proporcionan una base objetiva para la toma de decisiones. En el deporte, esto significa que desde la selección de jugadores hasta la estrategia de juego, las decisiones pueden ser informadas por análisis de datos en lugar de solo por intuición o experiencia pasada.

Técnicas y Herramientas de Análisis de Datos

Técnicas Estadísticas: Incluyen métodos básicos como medias, desviaciones estándar, y análisis de correlación, hasta técnicas más avanzadas como modelos predictivos y análisis de regresión.

Herramientas analíticas: varían desde software de estadística básica, como Excel, hasta plataformas más avanzadas o específicas del análisis deportivo.

2.1.3 Relevancia de la inteligencia artificial en el deporte

Transformación del entrenamiento y la estrategia

Cambios en la forma de entrenamiento: La inteligencia artificial (IA) está redefiniendo las técnicas de entrenamiento en el deporte. Utilizando análisis de datos avanzados, la IA permite a los entrenadores diseñar programas de entrenamiento que se adaptan específicamente a las necesidades y habilidades de cada jugador, mejorando así su rendimiento de manera más eficiente.

Mejoramiento de las tácticas de juego: En la estrategia de juego, la IA puede analizar patrones de juego, fortalezas y debilidades del equipo contrario, y proporcionar recomendaciones tácticas basadas en datos. Esto conduce a una planificación más estratégica y táctica en el campo.

Mejora en la detección y desarrollo de talentos

Identificación temprana de talentos: La IA está transformando la forma en que se identifican los talentos en el deporte. Al analizar datos de rendimiento desde etapas tempranas, los algoritmos de IA pueden identificar jugadores con alto potencial que quizás no sean evidentes a través de métodos tradicionales.

Desarrollo Personalizado de Jugadores: Mediante el análisis de datos detallados sobre las habilidades y el rendimiento de los jugadores, la IA puede ayudar en la creación de programas de desarrollo personalizados, asegurando que los jugadores reciban el entrenamiento más adecuado para sus necesidades específicas.

Desafíos y consideraciones éticas

Privacidad y Seguridad de los Datos: La implementación de IA en el deporte plantea preocupaciones sobre la privacidad y seguridad de los datos de los jugadores, especialmente cuando se manejan grandes volúmenes de información personal y de rendimiento.

Ética en el uso de datos: Es crucial asegurar que los datos se utilicen de manera ética y responsable, manteniendo la integridad y el respeto por los derechos de los jugadores.

Sesgos en algoritmos de IA: existe el riesgo de que los algoritmos incorporen sesgos inadvertidos, lo que podría conducir a decisiones injustas o inexactas. Por ello, los modelos deben desarrollarse y evaluarse con criterios de equidad, transparencia y revisión humana.

2.1.4 Ciencia de datos y Big Data

Data Science

La Ciencia de Datos es un campo interdisciplinario que emplea métodos científicos, procesos, algoritmos y sistemas para extraer conocimientos y discernimientos a partir de datos tanto estructurados como no estructurados. Este campo integra diversas áreas como la estadística, la informática y el aprendizaje automático, con el objetivo de analizar y comprender grandes volúmenes de datos. La meta principal de la Ciencia de Datos es transformar los datos en información útil que pueda

ser utilizada para la toma de decisiones estratégicas, optimización de procesos y descubrimiento de nuevas oportunidades de negocio.

Big Data

Big Data se refiere a conjuntos de datos extremadamente grandes y complejos que no pueden ser gestionados ni procesados por las herramientas tradicionales de gestión de datos. Estos datos se caracterizan por su volumen (gran cantidad de datos), velocidad (rapidez con la que se generan y procesan) y variedad (diferentes tipos de datos, como estructurados, no estructurados y semiestructurados). La gestión de Big Data requiere el uso de tecnologías avanzadas y técnicas analíticas para almacenar, procesar y analizar estos datos con el fin de descubrir patrones, tendencias y relaciones significativas que puedan proporcionar información valiosa para la toma de decisiones.

Data Science y Big Data en el deporte:

- Transformación del Scouting en la identificación de Talentos
- Predicción de Potencial
- Influencia en la Estrategia Deportiva
- Optimización de Estrategias
- Análisis en tiempo real. Mejora en la preparación física.
- Monitoreo de la condición física. Impacto en la toma de decisiones.
- Decisiones Basadas en Datos

2.1.5 Analítica de datos

Descripción: Los analistas de datos se centran en examinar, limpiar y procesar datos, generalmente para obtener información relevante que se pueda utilizar en la toma de decisiones. Su enfoque está más en el

análisis descriptivo que en la creación de modelos predictivos complejos.

Responsabilidades:

- Recopilar, organizar y limpiar datos.
- Analizar patrones, tendencias y correlaciones en los datos.
- Crear visualizaciones, informes y dashboards para reportar los hallazgos.
- Trabajar con herramientas de visualización (como Power BI) y lenguajes de consulta de datos (como SQL).

La analítica de datos y Big Data son conceptos relacionados y complementarios dentro de la ciencia de datos:

- Data Analytics se centra en examinar, procesar y extraer información significativa de los datos, generalmente a través de herramientas estadísticas y de visualización. Este análisis puede aplicarse a datos de cualquier tamaño, tanto a conjuntos de datos pequeños como a grandes volúmenes.

2.1.6 Tecnologías consideradas y utilizadas

SQL y motores relacionales (SQLite local; PostgreSQL en despliegue; SQL Server como alternativa futura)

Gestión de Bases de Datos: SQL se utiliza para gestionar bases de datos donde se almacenan grandes volúmenes de datos deportivos. Es esencial para realizar consultas eficientes, almacenamiento y recuperación de datos relacionados con jugadores, estadísticas de partidos, etc.

2.1.7 Fundamentos de modelado y evaluación

2.1.7.1 Redes neuronales feed-forward y clasificación binaria

Una red neuronal feed-forward transforma un vector de características en una salida numérica mediante capas densas y funciones de activación. En problemas de clasificación binaria, la salida puede interpretarse como un score que luego se transforma en probabilidad o categoría mediante un umbral definido.

2.1.7.2 Normalización y vector de características

La normalización permite llevar atributos heterogéneos a rangos comparables antes de ingresarlos al modelo. En scouting juvenil, los atributos pueden cargarse en una escala ordinal 1-20 y transformarse internamente a valores normalizados para el entrenamiento e inferencia.

2.1.7.3 Métricas de evaluación de clasificación

Para evaluar modelos de clasificación se emplean métricas complementarias. Accuracy mide aciertos globales, F1 combina precisión y recall, ROC-AUC evalúa separación entre clases y PR-AUC resulta especialmente útil cuando la clase positiva es minoritaria.

2.2 Trabajos relacionados y contexto de uso

Los trabajos relacionados pueden agruparse en cuatro líneas. PlayeRank propone una evaluación multidimensional y sensible al rol a partir de registros masivos de eventos de partido, y contrasta sus resultados con valoraciones de scouts profesionales (Pappalardo et al., 2019). El enfoque VAEP valora las acciones individuales según su efecto sobre las

probabilidades de marcar y recibir goles, incorporando el contexto de cada acción (Decroos et al., 2019).

En una línea más próxima al scouting predictivo, Lacan (2024) utiliza una arquitectura de aprendizaje profundo por stacking para detectar jugadores de alto potencial sobre una base abierta. Van Arem et al. (2025), por su parte, comparan modelos explicables para pronosticar la calidad y el valor futuro de futbolistas profesionales. TPScouting se diferencia de estos trabajos por su alcance: integra en un MVP académico la captura de datos, el seguimiento histórico, la visualización, las reglas operativas y la inferencia; utiliza datos sintéticos y no pretende competir con plataformas profesionales ni demostrar validez deportiva externa.

3. METODOLOGÍA

3.1 Enfoque de datos y modelado

El MVP utiliza datos sintéticos para validar el flujo completo de captura, persistencia, entrenamiento e inferencia. Esta decisión favorece la reproducibilidad y evita depender de bases privadas de clubes o proveedores externos durante la evaluación académica, pero limita la validez externa de los resultados.

3.1.1 Recolección y estructura de datos

Los datos se organizan alrededor de jugadores juveniles, atributos técnicos, físicos, mentales, estadísticas de rendimiento, disponibilidad, reportes de scouting y evaluaciones físicas. La edad operativa y la categoría juvenil se derivan de la fecha de nacimiento; por ejemplo, un jugador nacido en 2010 pertenece a la categoría 2010.

3.1.2 Preprocesamiento y normalización

Los atributos deportivos se cargan en escala 1-20. Para el modelo se transforman a representaciones numéricas compatibles con el preprocesador. Las posiciones se codifican como features categóricas y se incorporan variables históricas cuando existen registros previos.

3.1.3 Entrenamiento y evaluación

El entrenamiento se realiza con semilla fija, partición estratificada train/validation/test y métricas de clasificación. El modelo operativo es PlayerNet en PyTorch. Scikit-learn se utiliza para preprocesamiento, partición de datos, métricas, regresión logística de referencia y calibración isotónica; no constituye el motor principal de inferencia de la aplicación.

3.1.4 Definición de la variable objetivo

La corrida oficial no utiliza la columna `potential_label` como variable objetivo. El target efectivo es `temporal_target_label`, una etiqueta binaria sintética construida a partir de

señales futuras simuladas posteriores a un punto de corte: crecimiento y nivel futuro, rendimiento, dificultad o presión competitiva, consistencia, rol, disponibilidad, recuperación, evaluación scout y señales de consolidación o breakout. La clase positiva corresponde a los casos seleccionados por el score temporal, los controles de calidad y una cuota por cohorte de posición y edad.

El score previo a los controles pondera crecimiento (0,15), nivel futuro (0,12), rendimiento (0,13), presión (0,13), consistencia (0,10), rol (0,09), disponibilidad (0,09), recuperación (0,09), evaluación scout (0,10) y breakout (0,14), con una penalización por inestabilidad de 0,08. La distribución total registrada es de 1.597 casos positivos y 18.403 negativos sobre 20.000 jugadores sintéticos (7,985 %).

No se detectó una inclusión directa de la etiqueta o de datos futuros dentro de las variables de entrada. Sin embargo, los cuantiles y las cuotas utilizados para construir el target se calculan sobre el conjunto completo antes de dividir entrenamiento, validación y prueba. Esta decisión introduce un riesgo metodológico: las etiquetas de validación y prueba no son totalmente independientes de la distribución global, por lo que las métricas deben interpretarse como evidencia técnica preliminar.

3.2 Metodología de desarrollo

Se aplicó un enfoque incremental con prácticas inspiradas en Scrum, organizado en bloques pequeños y verificables. Cada cambio relevante se acompañó con pruebas focales, ejecución de la suite automatizada y documentación técnica para mantener trazabilidad. No se afirma una aplicación formal completa del marco Scrum, ya que no se documentaron todos sus roles, ceremonias y artefactos.

3.3 Ciclo de vida del software

1. Análisis de requisitos y alcance del MVP.
2. Diseño de entidades, rutas, vistas y flujo de datos.
3. Implementación iterativa en Flask, SQLAlchemy, Jinja2, Bootstrap, Chart.js y PyTorch.
4. Pruebas automatizadas y validaciones manuales de flujos críticos.

5. Despliegue en Render con Gunicorn y PostgreSQL administrado para demo académica.

6. Documentación de limitaciones, evidencia y oportunidades de mejora.

3.4 Desglose de tareas y cronograma consolidado

Fase	Bloque	Actividad	Estado
1	Análisis y requisitos	Definición de alcance, actores, entidades y criterios de éxito.	Completado
2	Backend y persistencia	Modelos SQLAlchemy, rutas Flask, migraciones manuales y validaciones.	Completado
3	Frontend y UX	Vistas Jinja2, Bootstrap, modales CRUD, dashboard, comparadores y responsive.	Completado
4	Machine Learning	Dataset sintético, entrenamiento, métricas, checkpoint y carga en inferencia.	Completado
5	Seguridad y robustez	CSRF, sesión, roles, secret obligatoria, rate limiting básico y errores controlados.	Completado para MVP
6	Testing y CI	pytest, pytest-cov, GitHub Actions, smoke visual opcional y smoke Render.	Completado
7	Documentación y entrega	Alineación del trabajo escrito con la app real y evidencia verificable.	Completado

TABLA 3-1. DESGLOSE DE TAREAS Y CRONOGRAMA CONSOLIDADO.

4. ANÁLISIS Y DISEÑO

4.1 Arquitectura del sistema

4.1.1 Arquitectura general, capas y componentes

El proyecto utiliza una arquitectura web full-stack con Flask y Bootstrap, integrando persistencia relacional, análisis de datos y aprendizaje automático para apoyar la evaluación de jugadores juveniles. Big Data se aborda como contexto conceptual y no como una infraestructura implementada en el MVP (Pallets Projects, s. f.; SQLAlchemy, s. f.).

- Backend:
 - Lenguaje: Python.
 - Framework: Flask.
 - Base de datos: SQLite para desarrollo local y PostgreSQL administrado en Render para la demo pública, ambos gestionados mediante SQLAlchemy (ORM). Se mantiene portabilidad hacia motores relacionales alternativos sin modificar la lógica de dominio principal.
 - Bibliotecas para Machine Learning: PyTorch y Scikit-learn.
 - Almacenamiento: En desarrollo local los datos se pueden almacenar en SQLite; en el despliegue verificado se utiliza PostgreSQL en Render. La estructura registra jugadores, atributos, evaluaciones, disponibilidad, rendimiento y reportes a lo largo del tiempo.
- Frontend:

- o Diseño de la interfaz: Basado en Bootstrap, con gráficos generados dinámicamente mediante Chart.js y tablas HTML con filtros y paginación.
- o Interacción del usuario: Los usuarios pueden consultar datos de jugadores, visualizar proyecciones y analizar evolución mediante gráficos interactivos y comparadores.

La Figura 4-1 resume los componentes principales de la solución implementada: cliente web, aplicación Flask, blueprints funcionales, servicios auxiliares, persistencia y módulo de aprendizaje automático.

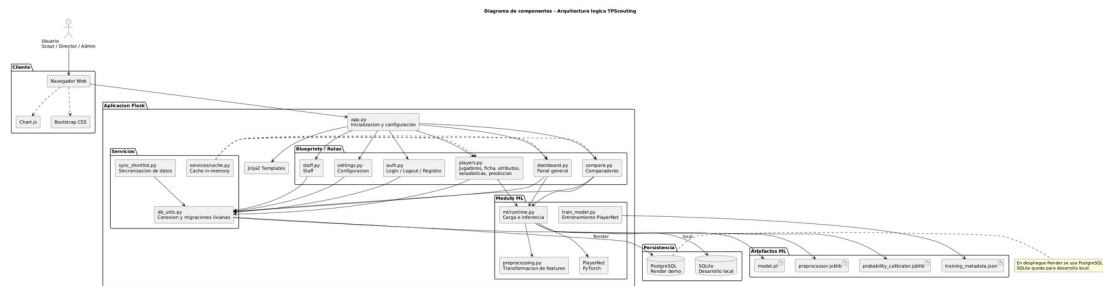


FIGURA 4-1. DIAGRAMA DE COMPONENTES DE TPSCOUTING.

Análisis de componentes específicos:

- Flask: es un framework web liviano y minimalista para Python que se utiliza para desarrollar aplicaciones web. Se describe como un microframework porque tiene un enfoque mínimo, permitiendo a los desarrolladores añadir solo los componentes que necesiten.

Características clave de Flask:

1. **Microframework:** Flask es un framework pequeño y flexible. No incluye herramientas complejas por defecto, lo que lo hace ideal para aplicaciones pequeñas o medianas, aunque también es lo suficientemente escalable para aplicaciones más grandes.
2. **Simplicidad:** Flask es fácil de entender y de usar, lo que lo hace muy atractivo para principiantes. Permite a los desarrolladores centrarse en escribir código sin preocuparse por demasiados detalles del framework.
3. **Extensible:** Aunque Flask tiene un núcleo ligero, se puede expandir fácilmente mediante extensiones. Estas extensiones permiten añadir funcionalidades como gestión de base de datos, autenticación, formularios, etc., sin que el framework pierda su simplicidad.

4. Flexibilidad: Flask no obliga a seguir una arquitectura o estructura estricta. Se puede organizar la aplicación de la manera que se prefiera. Esa flexibilidad brinda libertad a la hora de diseñar el código.
 5. Servidor de desarrollo incorporado: Flask viene con un servidor de desarrollo integrado, lo que permite probar la aplicación de forma local durante el desarrollo, sin la necesidad de configuraciones complejas.
- Flask se suele usar para:
 1. Aplicaciones web simples o medianas: Flask es ideal para desarrollar aplicaciones web pequeñas o medianas, como blogs, páginas personales, aplicaciones web empresariales, o APIs.
 2. API RESTful: Debido a su flexibilidad y simplicidad, Flask es ampliamente utilizado para crear APIs RESTful. Permite a los desarrolladores construir servicios que pueden ser consumidos por otros servicios o aplicaciones front-end.
 3. Prototipos: Flask es rápido de configurar y usar, por lo que es excelente para crear prototipos de aplicaciones web o validar ideas rápidamente.
 4. Integración con otras tecnologías: Se puede integrar fácilmente con bases de datos relacionales como SQL o no relacionales como MongoDB. Además, es compatible con motores de plantillas como Jinja2, que facilita la creación de interfaces de usuario dinámicas.

5. Desarrollo backend para aplicaciones SPA (Single Page Applications): Puede servir como el backend de aplicaciones de una sola página que utilizan frameworks como React, Vue.js o Angular en el front-end.

Comparación con otros frameworks:

- Flask es más ligero y flexible que Django, un framework más completo y con una estructura predeterminada. Mientras Django resulta apropiado para aplicaciones con mayores necesidades integradas de administración, Flask ofrece simplicidad y control explícito sobre los componentes.

Comparación: Flask (Python) vs. .NET Core Minimal APIs (C#)

Ahora, compararemos Flask con un framework similar en el ecosistema .NET, que es .NET Core Minimal APIs. Este enfoque de .NET es un equivalente en términos de simplicidad y ligereza a Flask, ya que también proporciona un marco básico para construir servicios web rápidos y minimalistas sin sobrecargar al desarrollador con funcionalidades complejas desde el principio.

4.1.1.1 Flask (Python)

1. Lenguaje: Python.
2. Enfoque: Es extremadamente minimalista y solo proporciona lo esencial para iniciar una aplicación web o API RESTful. La mayoría de las funcionalidades adicionales (como autenticación, manejo de formularios, etc.) se agregan mediante extensiones.

3. Flexibilidad: Flask te permite total flexibilidad en la estructura de tu aplicación. No hay convenciones estrictas y puedes estructurar el proyecto a tu manera.
4. Componentes opcionales: Se pueden elegir muchas extensiones para agregar funcionalidades como bases de datos, autenticación, etc.
5. Facilidad de uso: Fácil de aprender, lo que lo hace ideal para principiantes, y es muy rápido para prototipos y aplicaciones pequeñas/medianas.
6. Escalabilidad: Aunque es más adecuado para proyectos pequeños o medianos, Flask puede escalar a aplicaciones más grandes con el uso de extensiones y una buena estructura de proyecto.
7. Despliegue: Desplegar Flask es relativamente sencillo. Se puede ejecutar fácilmente en servidores estándar como Apache o Nginx con Gunicorn o uWSGI.
8. Popularidad: Muy popular en la comunidad Python y usado en muchos proyectos pequeños y medianos, incluyendo APIs.

4.1.1.2 .NET Core Minimal APIs (C#)

1. Lenguaje: C# (y otros lenguajes compatibles con .NET como F# o VB.NET).
2. Enfoque: Al igual que Flask, .NET Core Minimal APIs es un marco muy ligero para construir servicios web o APIs RESTful con un código mínimo. Introducido en .NET 6, este enfoque

proporciona una API muy simple para crear servicios web sin necesidad de usar el pesado modelo MVC.

3. Flexibilidad: Minimal APIs te permiten crear servicios web sin seguir la estructura estricta de un proyecto MVC tradicional. Puedes manejar rutas y controladores de una manera muy directa.
4. Componentes opcionales: .NET Minimal APIs soporta la integración con componentes opcionales, como Entity Framework Core (para manejo de bases de datos), autenticación de usuarios, etc., aunque no los incluye por defecto.
5. Facilidad de uso: Aunque C# y .NET son generalmente más complejos que Python, las Minimal APIs han simplificado bastante la creación de servicios web en el ecosistema .NET, haciendo que sea bastante fácil de aprender para los desarrolladores familiarizados con el lenguaje.
6. Escalabilidad: Muy adecuado para aplicaciones grandes debido a la robustez y el rendimiento de .NET. Incluso si comienzas con Minimal APIs, puedes escalar a aplicaciones más complejas con facilidad, y .NET Core ofrece un excelente rendimiento.
7. Despliegue: .NET Core es multiplataforma, por lo que las aplicaciones pueden ejecutarse en Windows, Linux y macOS, con soporte fácil para Docker y Kubernetes.
8. Popularidad: Cada vez más popular entre los desarrolladores C# por su simplicidad y el rendimiento que ofrece para crear

servicios rápidos. Es común en aplicaciones empresariales o servicios que necesitan alto rendimiento.

Comparación de Flask vs. .NET Core Minimal APIs:

Característica	Flask (Python)	.NET Core Minimal APIs (C#)
Lenguaje	Python	C# (.NET Core)
Simplicidad	Muy simple, ideal para principiantes	Simple para desarrolladores con experiencia en C#
Flexibilidad	Estructura flexible	Flexible, sin necesidad de usar MVC
Escalabilidad	Escalable, pero más adecuado para proyectos pequeños/medianos	Altamente escalable para proyectos grandes
Componentes opcionales	Muchas extensiones disponibles	Soporte opcional para Entity Framework, autenticación, etc.
Rendimiento	Bueno para la mayoría de las aplicaciones	Excelente rendimiento, especialmente en servicios de alto tráfico
Facilidad de despliegue	Fácil de desplegar en servidores estándares	Multiplataforma, soporte excelente para Docker y Kubernetes
Popularidad	Popular en el ecosistema Python	Popular en el ecosistema empresarial y de alto rendimiento

TABLA 4-1. COMPARACIÓN ENTRE FLASK Y .NET CORE MINIMAL APIS.

- Flask es una excelente opción si se prefiere trabajar en Python y se necesita algo ligero, flexible y fácil de usar para aplicaciones pequeñas o medianas.
 - .NET Core Minimal APIs es ideal para desarrolladores C# que buscan un enfoque minimalista pero poderoso para crear servicios web rápidos con la posibilidad de escalar a proyectos grandes y complejos, con un rendimiento sobresaliente.
- ➔ Para gestionar la persistencia de datos en el backend se utiliza SQLAlchemy ORM con integración directa en Flask, lo que abstrae el acceso al motor relacional y permite operar con SQLite local y PostgreSQL en Render manteniendo portabilidad.

- o SQLAlchemy ORM en Flask: en esta implementación se configura mediante `create_engine` y `sessionmaker`, manteniendo control explícito de sesiones, consultas y transacciones sobre una base relacional.

Características clave de SQLAlchemy (uso actual en el MVP):

1. ORM (Object-Relational Mapping): SQLAlchemy permite interactuar con bases de datos utilizando clases y objetos en lugar de escribir consultas SQL manualmente. Esto hace que el código sea más limpio, mantenible y trazable.
2. Soporte para varias bases de datos: SQLAlchemy soporta motores populares como PostgreSQL, MySQL, SQLite, Oracle, entre otros. Esto facilita migraciones futuras con cambios controlados de configuración.
3. Consulta fluida: SQLAlchemy proporciona un sistema de consultas fluido y expresivo, lo que permite escribir consultas complejas de una manera legible.
4. Gestión de relaciones: Soporta relaciones entre tablas, como one-to-many, many-to-many, y one-to-one, lo cual es útil para manejar estructuras de datos más complejas.
5. Migraciones con Flask-Migrate: A menudo se usa junto con Flask-Migrate, que permite gestionar los cambios en el esquema de la base de datos (migraciones) de manera sencilla utilizando el sistema de migraciones de Alembic. Esto es útil para mantener sincronizada la base de datos con el código a medida que evoluciona la aplicación.

6. Integración sencilla con Flask: La configuración es muy simple dentro de Flask, generalmente solo necesitas definir la cadena de conexión de la base de datos en tu configuración y vincular la instancia de SQLAlchemy a tu aplicación Flask.

Comparación de SQLAlchemy con otro ORM: Peewee

Peewee es otro ORM popular en Python que suele utilizarse en proyectos que buscan una opción más liviana. En este trabajo, SQLAlchemy se prioriza por robustez y escalabilidad dentro del MVP.

Característica	SQLAlchemy ORM (en Flask)	Peewee
Popularidad	Muy popular, ampliamente utilizado en proyectos grandes y medianos.	Menos popular pero usado en proyectos pequeños y medianos.
Complejidad	Moderadamente complejo, pero flexible. Permite consultas avanzadas y soporte para grandes aplicaciones.	Más simple y ligero. Adecuado para proyectos más pequeños o medianos que no requieren tanta flexibilidad.
Soporte para bases de datos	Soporta una amplia variedad de bases de datos (PostgreSQL, MySQL, SQLite, Oracle, etc.).	Soporta las mismas bases de datos populares, pero es más ligero.
Sistema de migraciones	Utiliza Flask-Migrate junto con Alembic para gestionar migraciones.	Peewee tiene su propio sistema de migraciones (peewee-migrate), pero no es tan completo como Alembic.
Relaciones entre tablas	Soporta relaciones complejas como one-to-one, one-to-many, many-to-many.	También soporta relaciones, pero Peewee está más limitado en consultas muy complejas.
Facilidad de uso	Puede ser más difícil de configurar inicialmente debido a su flexibilidad y a la complejidad del ORM.	Muy fácil de configurar, especialmente para proyectos pequeños.
Rendimiento	Ideal para aplicaciones más grandes y complejas que pueden necesitar optimización de consultas SQL.	Es más ligero, por lo que puede ser más rápido en proyectos más pequeños y con menos carga de trabajo.

TABLA 4-2. COMPARACIÓN ENTRE SQLALCHEMY Y PEWEE.

Se adopta SQLAlchemy porque ofrece:

- Robustez, para proyectos medianos y grandes.

- Escalabilidad: ante una posible evolución y aumento de complejidad de la aplicación, se requiere una solución capaz de acompañar el crecimiento del dominio.
- Flexibilidad: Ya que se desea tener control completo sobre las consultas y cómo interactúas con la base de datos.

Cuándo elegiríamos Peewee:

- Proyectos pequeños: proyecto pequeño y se prefiere una solución más ligera,
- Simplicidad: Si no se necesitan todas las funcionalidades avanzadas que ofrece SQLAlchemy, Peewee puede resultar una interfaz más sencilla y menos sobrecargada.
- Rapidez en la configuración: Si se desea levantar una aplicación rápidamente con un ORM sencillo

Conclusión técnica sobre el ORM adoptado:

- SQLAlchemy es un estándar de facto en proyectos Flask por su flexibilidad, potencia y capacidad de escalar. Es adecuado cuando el proyecto requiere consultas avanzadas, relaciones complejas y evolución progresiva del dominio. Peewee, en cambio, puede ser una alternativa liviana para escenarios más pequeños.

La arquitectura actual es una aplicación Flask server-side con Jinja2 y Bootstrap en frontend, SQLAlchemy como ORM, SQLite para desarrollo local y PostgreSQL en Render para demo pública. El

backend fue modularizado parcialmente mediante blueprints por familia: auth, staff, players, dashboard, compare y settings. La inferencia ML se encapsula en módulos de runtime, manteniendo compatibilidad con los endpoints existentes.

Capa	Archivos principales	Responsabilidad
Presentación	templates/, static/	Vistas Jinja2, Bootstrap, Chart.js, CSS propio
Rutas	scouting_app/routes/	Blueprints por familia funcional
Servicios	scouting_app/services/	Cache, seguridad, locks y datos operativos
Dominio	models.py, player_logic.py	Entidades SQLAlchemy y reglas de negocio
ML	train_model.py, ml/runtime.py	Entrenamiento, checkpoint, carga e inferencia
Persistencia	SQLite/PostgreSQL	SQLite local; PostgreSQL en Render

TABLA 4-4. CAPAS Y RESPONSABILIDADES DEL MVP.

4.1.2 Diseño de la Base de datos

- La base de datos se diseñó de forma relacional. En el MVP se implementa en SQLite para manejar grandes cantidades de datos históricos relacionados con jugadores juveniles.
- Tablas principales:
 1. players (Player): almacena información del jugador (identificador, datos personales, posición, club/país, atributos técnicos/físicos/mentales y etiqueta de potencial).
 2. player_stats (PlayerStat): registra estadísticas por fecha (partidos/microciclos), incluyendo minutos, goles, asistencias y score final.

3. `player_attribute_history` (`PlayerAttributeHistory`): conserva el historial técnico por fecha para analizar evolución de atributos.

- Relaciones: `Player` se vincula en relación 1:N con `PlayerStat` y 1:N con `PlayerAttributeHistory`, permitiendo trazabilidad longitudinal por jugador.

Análisis de componente específico:

Característica	SQL Server	MySQL	PostgreSQL
Escalabilidad	Altamente escalable con particionamiento avanzado	Bueno, pero con menos opciones avanzadas	Excelente, también altamente escalable
Seguridad	Encriptación avanzada, auditoría, soporte empresarial	Seguridad básica (sin encriptación por defecto)	Muy buen nivel de seguridad, buena auditoría
Rendimiento en consultas complejas	Muy bueno, especialmente para grandes volúmenes de datos	Muy bueno, pero menos optimizado en consultas complejas	Excelente rendimiento en consultas complejas
Soporte para Python	Integración con Machine Learning Services y PyODBC	PyMySQL o MySQL Connector para Python	Psycopg2 para PostgreSQL, excelente integración
Coste	Puede tener licencia dependiendo del uso	Gratuito (open source)	Gratuito (open source)

TABLA 4-3. COMPARACIÓN ENTRE MOTORES RELACIONALES.

En desarrollo local se utiliza SQLite por su simplicidad y bajo costo operativo. En el despliegue de Render se utiliza PostgreSQL administrado para evitar depender de archivos SQLite en un filesystem efímero. SQLAlchemy permite mantener el mismo modelo de dominio en ambos entornos.

Para una producción real con más usuarios, concurrencia o auditoría, se recomienda sostener PostgreSQL u otro motor administrado, junto con backups, monitoreo y migraciones formales. Para el MVP

académico se documenta la diferencia entre entorno local y despliegue.

4.1.3 Interfaz de Usuario

- La interfaz fue desarrollada usando Bootstrap para asegurar un diseño limpio, responsivo y fácil de navegar.
- Una Landing Page con un login, que permite a los usuarios ingresar al sistema y acceder al panel de control donde pueden gestionar jugadores, ver estadísticas y generar proyecciones.
- Panel de Jugadores: Donde los usuarios pueden ver la lista de jugadores y acceder a su información detallada, incluyendo sus atributos y sus evaluaciones.
- Gráficos y Tablas:
 - Chart.js permite crear gráficos dinámicos e interactivos, visualizando la evolución de los atributos del jugador.
 - Tablas HTML con filtros y paginación en backend para visualizar la información tabular de forma clara y operativa.

4.2 Funcionalidades y componentes

4.2.1 Módulos funcionales implementados

- Autenticación, registro de usuarios y control básico por roles.
- Gestión de jugadores, ficha completa, edición, baja y carga masiva CSV.
- Historial de atributos, rendimiento, partidos, disponibilidad, evaluaciones físicas y reportes scout.

- Panel general dinámico por rol: mesa de scouting para scout/administrador y estado de plantel para director técnico.
- Comparador 1v1 y comparador múltiple con métricas visuales.
- Predicción de potencial y recomendaciones operativas en ficha de jugador.

4.2.2 Modelo de evaluación deportiva

La evaluación del jugador combina datos cuantitativos y cualitativos. Los atributos principales se registran en escala 1-20 y se agrupan en dimensiones técnicas, físicas, mentales y defensivas. El sistema no reemplaza el criterio del scout; aporta orden, trazabilidad y comparación consistente entre jugadores.

Dimensión	Variables representativas	Fuente
Técnica	pace, shooting, passing, dribbling, technique, vision	Observación técnica y carga de atributos
Física	physical, stamina, strength, agility, pace	Evaluaciones y reportes
Defensiva	defending, tackling, marking	Perfil por posición
Mental	determination, decision_making, work_rate, composure	Reportes y atributos
Rendimiento	minutos, goles, asistencias, precisión, puntaje final	Historial de partidos/entrenamientos

TABLA 4-5. DIMENSIONES DE EVALUACIÓN DEPORTIVA.

4.2.3 Uso de PyTorch y Scikit-learn

PyTorch se emplea para implementar PlayerNet, el modelo cargado por el backend para inferencia. Scikit-learn se utiliza como soporte metodológico para el preprocesamiento, la partición de datos, las métricas, la regresión logística de referencia y la calibración isotónica. No se implementa una regresión lineal productiva para la inferencia web.

4.2.3.1 Síntesis explicativa del flujo predictivo

En términos simples, PlayerNet es una red neuronal propia del proyecto que recibe 68 variables del jugador y devuelve una probabilidad cruda entre 0 y 1. Esa salida expresa el

grado en que el caso se parece a los ejemplos positivos aprendidos durante el entrenamiento; no equivale, por sí sola, a una certeza deportiva.

La probabilidad calibrada es un reajuste posterior de la salida cruda mediante regresión isotónica. Su objetivo es mejorar la interpretación numérica de la probabilidad, no garantizar mejores valores en todas las métricas. Por eso una versión calibrada puede ser más prudente y, al mismo tiempo, obtener un ROC-AUC, PR-AUC o F1 levemente inferior.

La aplicación también calcula `combined_prob`, un score operativo que combina la probabilidad cruda de PlayerNet, el promedio histórico del jugador y el ajuste por posición con pesos 0,35, 0,35 y 0,30. Cuando falta algún componente, los pesos disponibles se renormalizan. Este score es el valor integrado utilizado por la interfaz, pero no fue evaluado de manera independiente en el conjunto de prueba.

La regresión logística cumple el papel de modelo de referencia. En la corrida documentada, su desempeño fue muy similar al de PlayerNet: obtuvo valores levemente superiores en ROC-AUC y F1, mientras que PlayerNet crudo alcanzó el mayor PR-AUC. En consecuencia, el trabajo no sostiene una superioridad global de la red neuronal; la conclusión es que una arquitectura más compleja no garantiza mejores resultados y debe seleccionarse según evidencia.

4.2.4 Seguridad y autenticación

El MVP implementa autenticación por usuario y contraseña, contraseñas hasheadas con Werkzeug, sesiones firmadas por Flask, rutas protegidas, roles básicos y validación CSRF en formularios POST que modifican datos. En producción/Render la variable `APP_SECRET_KEY` es obligatoria y la aplicación no debe ejecutarse con claves por defecto (OWASP Foundation, 2021).

El login incorpora rate limiting en memoria por proceso. Esta mitigación es suficiente para una demo académica, pero no equivale a un limitador distribuido para producción multi-instancia.

4.3 Requisitos del sistema

ID	Requisito	Estado
RF-01	Autenticación por sesión para acceso a la aplicación.	Implementado
RF-02	Gestión de usuarios con roles administrador, scout y director.	Implementado
RF-03	Alta, edición y baja de jugadores.	Implementado
RF-04	Ficha de jugador con perfil, edad, categoría, atributos, historial y predicción.	Implementado
RF-05	Registro de atributos en escala 1-20 con historial.	Implementado
RF-06	Registro de rendimiento, partidos, disponibilidad, físico y reportes.	Implementado
RF-07	Cálculo de potencial con PlayerNet en PyTorch.	Implementado
RF-08	Clasificación de potencial: bajo <60%, medio 60-79%, alto >=80%.	Implementado
RF-09	Carga masiva CSV compatible con Excel y validación por fila.	Implementado
RF-10	Panel general con métricas accionables por rol.	Implementado
RF-11	Comparadores 1v1 y múltiple.	Implementado
RF-12	Healthcheck y deploy en Render.	Implementado

TABLA 4-6. REQUISITOS FUNCIONALES DEL SISTEMA.

4.4 Modelo de datos

El modelo de datos se implementa con SQLAlchemy. La entidad central es Player, vinculada a historiales de atributos, estadísticas, partidos, disponibilidad, evaluaciones físicas y reportes scout. La fecha de nacimiento es la fuente para calcular edad actual y categoría juvenil.

Entidad	Propósito
Player	Jugador, datos personales, posición, club, atributos base, birth_date, photo_url, potencial.
PlayerAttributeHistory	Historial de atributos técnicos/físicos/mentales en escala 1-20.
PlayerStat	Estadísticas de rendimiento por fecha.

Entidad	Propósito
Match / PlayerMatchParticipation	Contexto de partidos y participación del jugador.
PhysicalAssessment	Evaluación física y mediciones asociadas.
PlayerAvailability	Disponibilidad, fatiga y estado físico.
ScoutReport	Reporte cualitativo del scout.
Coach / Director / User	Staff, directivos y usuarios del sistema.

TABLA 4-7. ENTIDADES PRINCIPALES DEL MODELO DE DATOS.

La Figura 4-2 muestra las clases principales del modelo de dominio persistido con SQLAlchemy y sus relaciones mas relevantes.

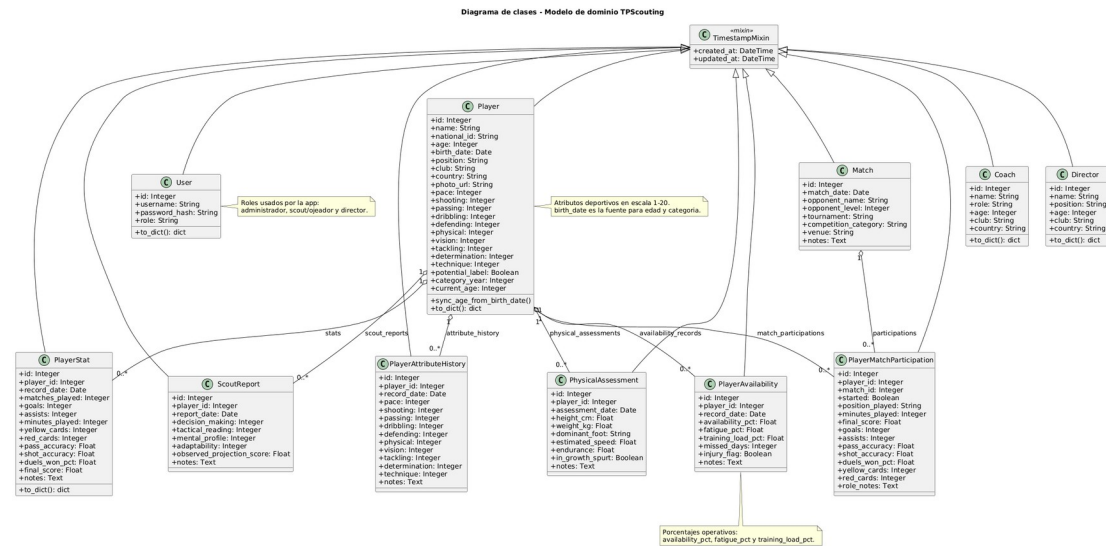


FIGURA 4-2. DIAGRAMA DE CLASES DEL MODELO DE DOMINIO.

4.5 Endpoints principales del MVP

Método	Ruta	Propósito
GET/POST	/login	Autenticación.
POST	/logout	Cierre de sesión con CSRF.
GET/POST	/register	Alta de usuarios según permisos.
GET	/dashboard	Panel general dinámico por rol.
GET	/players	Listado paginado y filtrable de jugadores.
GET/POST	/players/manage	Alta individual de jugador.
GET/POST	/players/import	Carga masiva CSV con previsualización.
GET/POST	/player/<id>/stats	Visualización y carga de rendimiento.
GET/POST	/player/<id>/attributes	Visualización y carga de atributos.
GET	/player/<id>/predict	Vista de proyección de potencial.
GET/POST	/compare	Comparador 1v1.
GET/POST	/compare/multi	Comparador múltiple.
GET/POST	/settings	Operaciones administrativas y pipeline.
GET	/health	Estado de servicio, base y calidad de datos.

TABLA 4-8. ENDPOINTS PRINCIPALES DEL MVP.

4.6 Decisiones tecnológicas y alineación con el MVP

Tecnología	Justificación
Flask	Permite desarrollo web server-side simple, auditable y suficiente para un MVP académico.
SQLAlchemy	Abstrae SQLite local y PostgreSQL en Render sin reescribir las reglas del dominio.
PyTorch	Permite implementar PlayerNet y guardar checkpoints con metadatos de entrada.
Scikit-learn	Aporta métricas, partición, modelo de referencia y calibración.
Bootstrap/Jinja2	Facilita una interfaz funcional y adaptable sin incorporar una SPA compleja.
Chart.js	Permite integrar visualizaciones ligeras en las vistas HTML.
pytest/pytest-cov	Proporciona validación automatizada y medición de cobertura en integración continua.
Render/Gunicorn	Permite un despliegue público para validación académica mediante HTTPS.

TABLA 4-9. DECISIONES TECNOLÓGICAS.

4.7 Casos de uso

ID	Caso de uso	Precondición	Resultado
CU-01	Iniciar sesión	Usuario registrado	Sesión activa y acceso según rol.
CU-02	Registrar jugador	Scout o administrador	Jugador persistido con fecha de nacimiento, edad y categoría.
CU-03	Cargar atributos	Jugador existente	Historial actualizado en escala 1-20.
CU-04	Cargar rendimiento/reportes	Jugador existente	Datos operativos visibles en ficha y dashboard.
CU-05	Calcular potencial	Modelo cargado y jugador con datos	Probabilidad, categoría y sugerencias visibles.
CU-06	Comparar jugadores	Dos o más jugadores seleccionados	Tabla/gráfico comparativo.
CU-07	Revisar panel general	Usuario autenticado	KPIs y listas accionables por rol.

TABLA 4-10. CASOS DE USO PRINCIPALES.

La Figura 4-3 resume los casos de uso principales del sistema, agrupando los roles de administrador, scout, director y el sistema de Machine Learning.

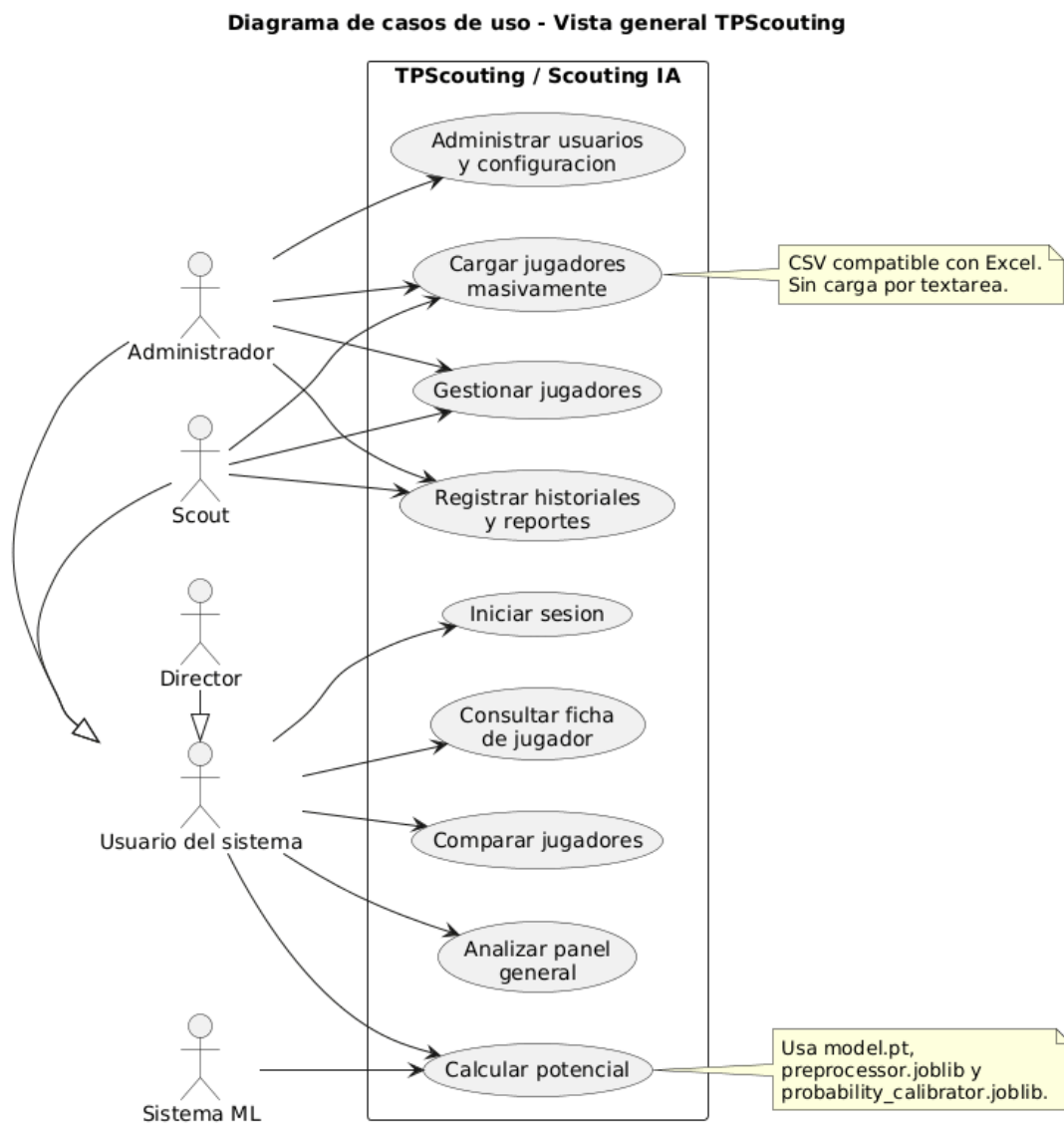


FIGURA 4-3. DIAGRAMA DE CASOS DE USO GENERAL.

La Figura 4-4 detalla los casos de uso asociados a alta, edición, consulta, carga masiva e historiales de jugadores.

Diagrama de casos de uso - Gestion de jugadores

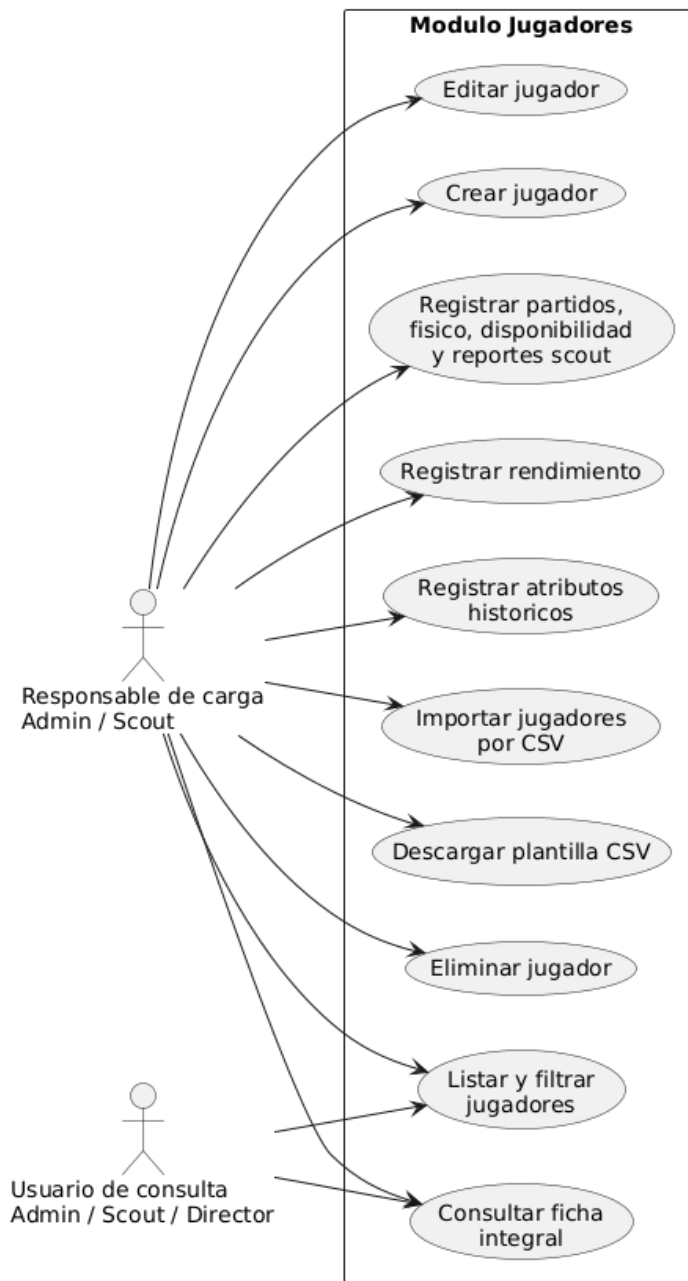


FIGURA 4-4. DIAGRAMA DE CASOS DE USO DE GESTIÓN DE JUGADORES.

La Figura 4-5 muestra los casos de uso orientados a consulta, comparación, panel general y predicción de potencial.

Diagrama de casos de uso - Analisis y decision scout

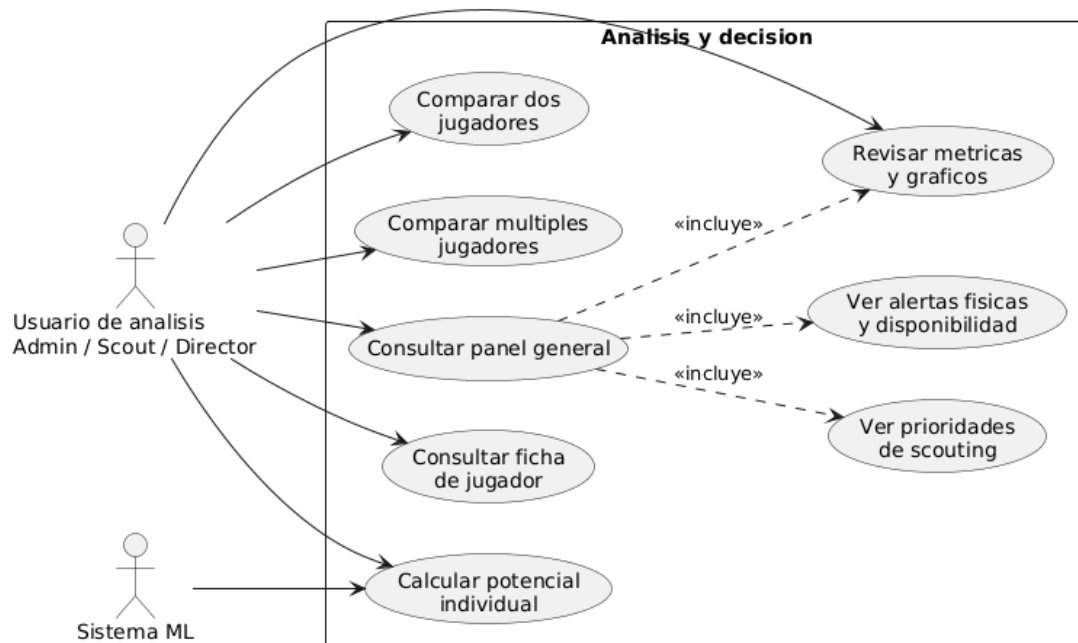


FIGURA 4-5. DIAGRAMA DE CASOS DE USO DE ANÁLISIS Y DECISIÓN SCOUT.

5. DESARROLLO

5.1 Implementación del backend

El backend se implementó en Flask con rutas server-side y blueprints por familia funcional. `app.py` conserva la inicialización, configuración, registro de blueprints, healthcheck, carga de artefactos ML y compatibilidad con endpoints históricos. Las rutas específicas se distribuyen en `scouting_app/routes/auth.py`, `staff.py`, `players.py`, `dashboard.py`, `compare.py` y `settings.py`.

Se incorporaron servicios auxiliares para seguridad, cache, locks y datos operativos. Esta separación mejora la mantenibilidad sin convertir el MVP en una arquitectura de microservicios innecesaria para el alcance académico.

La integración con PyTorch se mantiene dentro del backend Flask. Los artefactos `model.pt`, `preprocessor.joblib` y `probability_calibrator.joblib` se cargan al iniciar la aplicación y se validan contra `input_dim` para evitar incompatibilidades entre entrenamiento e inferencia.

Flujo técnico de inferencia: Flask recupera el jugador mediante SQLAlchemy; el preprocesador transforma edad, posición, atributos e historial en un vector de 68 variables; PlayerNet produce logits y la función sigmoid obtiene la probabilidad cruda. La calibración isotónica se utiliza en la evaluación experimental. Para la interfaz, la aplicación calcula `combined_prob` mediante un promedio ponderado de la probabilidad cruda del modelo (0,35), el promedio histórico (0,35) y el ajuste posicional (0,30), con renormalización cuando falta algún componente.

La Figura 5-1 detalla el flujo de inferencia cuando un usuario consulta la proyección de potencial de un jugador.

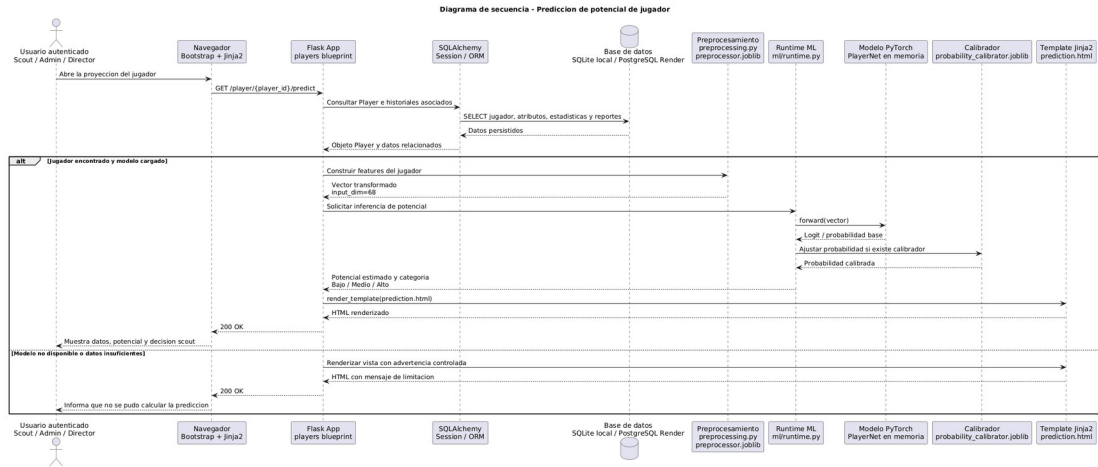


FIGURA 5-1. SECUENCIA DE PREDICCIÓN DE POTENCIAL DE JUGADOR.

5.2 Implementación del frontend

El frontend utiliza Jinja2, Bootstrap, CSS propio y Chart.js. La UX fue ajustada para panel general, fichas de jugador, comparadores, formularios y modales CRUD. La carga de registros históricos se separó visualmente de la lectura de datos mediante modales/offcanvas, evitando formularios extensos incrustados en la pantalla principal.

La Figura 5-2 describe la carga del panel general, incluyendo validación de sesión, caché en memoria, consultas SQLAlchemy y renderizado de la vista.

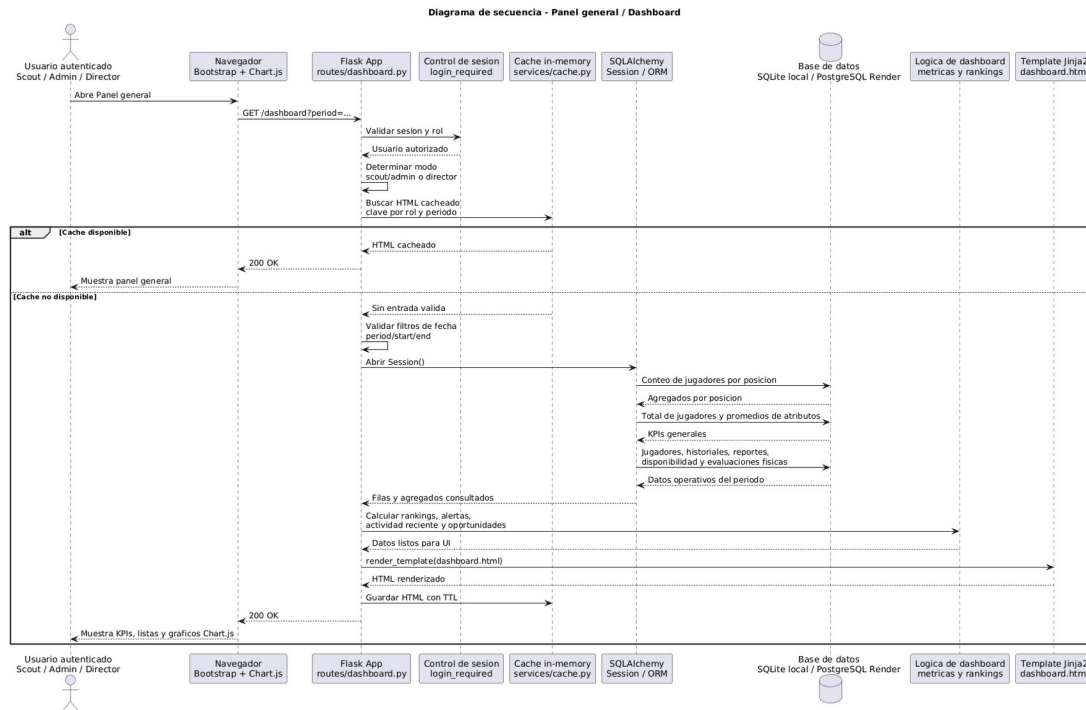


FIGURA 5-2. SECUENCIA DE CARGA DEL PANEL GENERAL.

5.3 Datos, edad y categoría juvenil

La fecha de nacimiento se consolidó como fuente de verdad. A partir de `birth_date` se calcula la edad actual y la categoría juvenil visible como “Cat. YYYY”. Para datos demo heredados sin fecha se generaron fechas determinísticas que conservan edades entre 12 y 18 años, dejando documentado que son datos sintéticos de MVP.

5.4 Entrenamiento del modelo de aprendizaje automático

El modelo operativo es PlayerNet. Su arquitectura combina una rama lineal amplia de 68 entradas a una salida y una rama residual no lineal de 68 a 128, 64 y 1 neurona, con BatchNorm, GELU y dropout de 0,15. El checkpoint contiene 17.994 parámetros entrenables. El entrenamiento utiliza BCEWithLogitsLoss con ponderación de la clase positiva, optimizador AdamW y una tasa de aprendizaje inicial de 0,0005.

Elemento	Valor verificado
Clase	PlayerNet
Variable objetivo	temporal_target_label

Elemento	Valor verificado
Dataset y partición	20.000 sintéticos: 14.000 train, 3.000 validation y 3.000 test
Distribución positiva	1.597/20.000 (7,985 %); test 240/3.000 (8 %)
input_dim	68 = 11 variables base + 52 históricas + 5 de posición
Arquitectura	Rama lineal 68->1 + rama residual 68->128->64->1; BatchNorm, GELU y dropout 0,15
Parámetros entrenables	17.994
Loss	BCEWithLogitsLoss con pos_weight=11,5224
Optimizador	AdamW; weight decay 0,0005
Learning rate / batch size	0,0005 / 256
Épocas solicitadas / ejecutadas	45 / 15
Mejor época / early stopping	Época 5; patience=10; monitor PR-AUC calibrado de validation y F1 como desempate
Training loss	0,6431 en mejor época; 0,5179 en última época
Validation loss / duración	No registradas en la corrida
Calibración	Regresión isotónica; threshold 0,25

TABLA 5-1. CONFIGURACIÓN VERIFICADA DEL MODELO PLAYERNET.

La selección del modelo no se presenta como una superioridad automática del aprendizaje profundo. La regresión logística balanceada y PlayerNet obtuvieron desempeños similares, con ventajas diferentes según la métrica. Por eso la regresión logística se conserva como modelo de referencia obligatorio y PlayerNet como modelo operativo integrado al MVP.

El pipeline registra metadata junto con el modelo: versión del checkpoint, input_dim, semilla, partición, historial de training loss, métricas de validación y test, baselines y artefactos generados. Se solicitaron 45 épocas; la mejor validación se alcanzó en la época 5 y el entrenamiento se detuvo en la época 15 después de diez épocas sin mejora, de acuerdo con patience=10. La corrida no conserva validation loss, duración total ni el SHA Git exacto del entrenamiento, por lo que esos datos se declaran como no disponibles.

5.5 Seguridad y robustez implementada

- APP_SECRET_KEY obligatoria en producción/Render.
- Protección CSRF en formularios POST mutantes.

- Logout por POST con CSRF.
- Passwords hasheadas con Werkzeug y validación mínima para usuarios nuevos.
- Rate limiting básico de login en memoria.
- Bloqueo de SQLite en producción salvo opt-in explícito.
- Healthcheck con validación de base y calidad mínima de datos.

5.6 Despliegue del sistema

El MVP se desplegó en Render con Gunicorn y PostgreSQL administrado. Por limitación del plan Free se utiliza una sola base PostgreSQL para la demo. La aplicación se valida mediante /health, login, panel general, listado, comparadores y vistas de jugador.

Elemento	Estado
URL pública	https://tpscouting-mvp.onrender.com
Servidor WSGI	Gunicorn --workers 1 --threads 2
Base en deploy	PostgreSQL administrado Render Free
Datos demo	100 jugadores con birth_date, national_id y photo_url completos
Modelo	Artefactos versionados: model.pt, preprocessor.joblib, probability_calibrator.joblib

TABLA 5-2. ESTADO DEL DESPLIEGUE EN RENDER.

En Render se evita SQLite como almacenamiento productivo debido al filesystem efímero. El despliegue utiliza PostgreSQL administrado, variables de entorno, Gunicorn con un worker y dos threads, y un seed demo idempotente para asegurar que la demo pública arranque con datos verificables.

La Figura 5-3 representa el despliegue verificado del MVP en Render con Gunicorn, Flask, PostgreSQL y artefactos de Machine Learning.

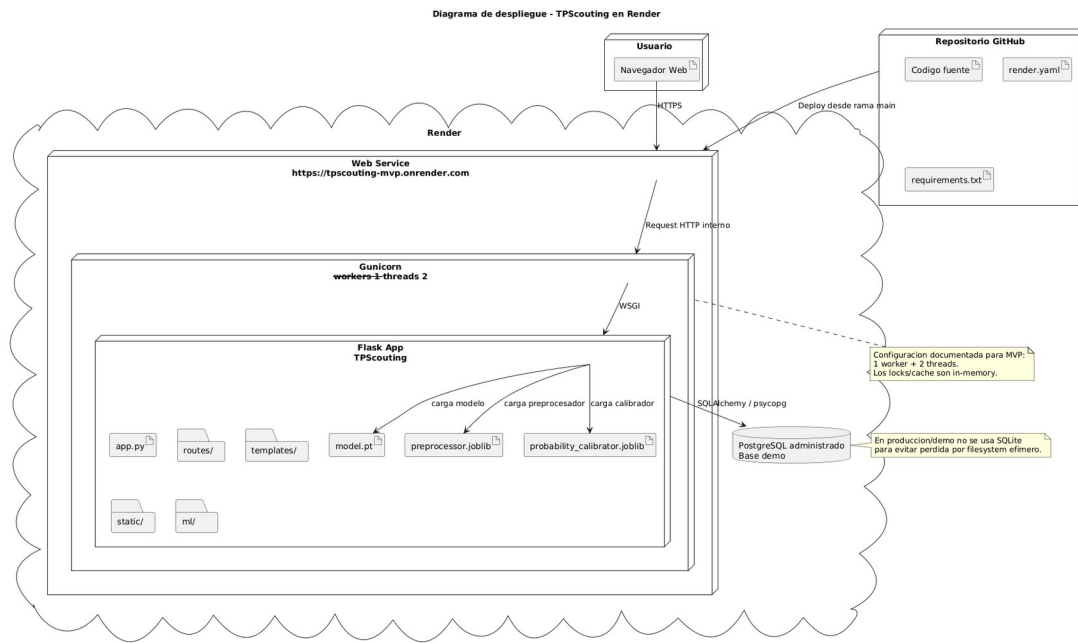


FIGURA 5-3. DIAGRAMA DE DESPLIEGUE EN RENDER.

5.7 Problemas técnicos y decisiones de implementación

Durante el desarrollo se resolvieron problemas que condicionaban la reproducibilidad y el despliegue. Las principales decisiones verificadas fueron las siguientes:

- Persistencia en Render: se reemplazó SQLite como almacenamiento del despliegue por PostgreSQL administrado, debido al sistema de archivos efímero del servicio.
- Alineación entre entrenamiento e inferencia: se persisten el preprocesador y el checkpoint, y el runtime valida la versión y el input_dim antes de utilizar el modelo.
- Concurrencia del pipeline: se incorporaron bloqueos de hilo y archivo y se configuró un único worker en la demo. Esta solución no equivale a un lock distribuido.
- Rendimiento de la demo: se incorporaron caché con TTL, límite de entradas y paginación. El arranque frío de Render Free continúa siendo una limitación.
- Datos juveniles: birth_date se consolidó como fuente de verdad para edad y categoría; las fechas generadas para datos heredados son sintéticas y no deben interpretarse como información real.

6. PRUEBAS Y RESULTADOS

6.1 Plan de pruebas

El plan de pruebas cubre flujos funcionales, seguridad, consistencia de datos, aprendizaje automático, panel general, comparadores, carga masiva, historiales y comportamiento sin modelo cargado. La suite automatizada se ejecuta con pytest; pytest-cov mide cobertura y GitHub Actions prueba Python 3.11 y 3.12.

Tipo	Cobertura	Evidencia
Funcional	Login, logout, rutas protegidas, jugadores, staff, comparadores, dashboard.	pytest
Seguridad	CSRF, roles, password mínima, secret obligatoria, rate limiting login.	pytest
Datos	birth_date, categoría, escala 1-20, migraciones manuales y validaciones.	pytest
ML	checkpoint, input_dim, métricas, warnings y compatibilidad legacy.	pytest
UI/Smoke	Render real y Playwright opcional.	Smoke HTTP / capturas

TABLA 6-1. COBERTURA DEL PLAN DE PRUEBAS.

6.2 Resultados del MVP

La ejecución local directa del 10/07/2026 obtuvo 83 pruebas aprobadas, 1 omitida y 4 advertencias en 38,88 segundos. La ejecución con pytest-cov obtuvo 79 % de cobertura sobre 5.082 sentencias, con 1.062 no cubiertas. La prueba omitida corresponde al smoke visual de Playwright, configurado como opt-in; las advertencias son RuntimeWarning de scikit-learn por columnas completamente NaN en dos pruebas de preprocesamiento.

Evidencia	Resultado verificado
pytest local	83 passed, 1 skipped, 4 warnings; 38,88 s
pytest-cov	79 %; 5.082 sentencias; 1.062 no cubiertas; 55,79 s
Test omitido	Smoke visual Playwright opt-in; requiere RUN_PLAYWRIGHT=1
Warnings	4 All-NaN slice encountered en dos tests de preprocesamiento
CI pública	Run #71 completada con éxito en Python 3.11 y 3.12
Commit certificado por CI	fa8a50d1173f2760329a45f11fbf93a709721235
Artefactos CI	coverage-python-3.11 y coverage-python-3.12

Evidencia	Resultado verificado
Alcance	La run pública no certifica el HEAD local 7d7680e, que estaba un commit por delante
Smoke Render	Validación histórica del 20/05/2026; control del 10/07/2026 finalizó por timeout

TABLA 6-2. RESULTADOS DE VALIDACIÓN AUTOMATIZADA Y CI.

6.2.1 Métricas de PlayerNet y calibración

La corrida registrada el 19/05/2026 utiliza semilla 42, 20.000 jugadores sintéticos y una partición de 14.000/3.000/3.000. La clase positiva representa el 8 % del conjunto de prueba. Las métricas se presentan por separado para la salida sigmoid cruda de PlayerNet y para la salida ajustada mediante calibración isotónica; ninguna de ellas corresponde al score combinado mostrado por la aplicación.

Métrica	Calibrado isotónico	Sigmoid crudo
Threshold	0,250	0,825
Accuracy	0,9303	0,9300
ROC-AUC	0,9174	0,9203
PR-AUC	0,5241	0,5461
F1	0,5282	0,5291
Precision	0,5764	0,5728
Recall	0,4875	0,4917
Matriz de confusión	[[2674, 86], [123, 117]]	[[2672, 88], [122, 118]]

TABLA 6-3. MÉTRICAS DE PLAYERNET CALIBRADO Y CRUDO EN TEST.

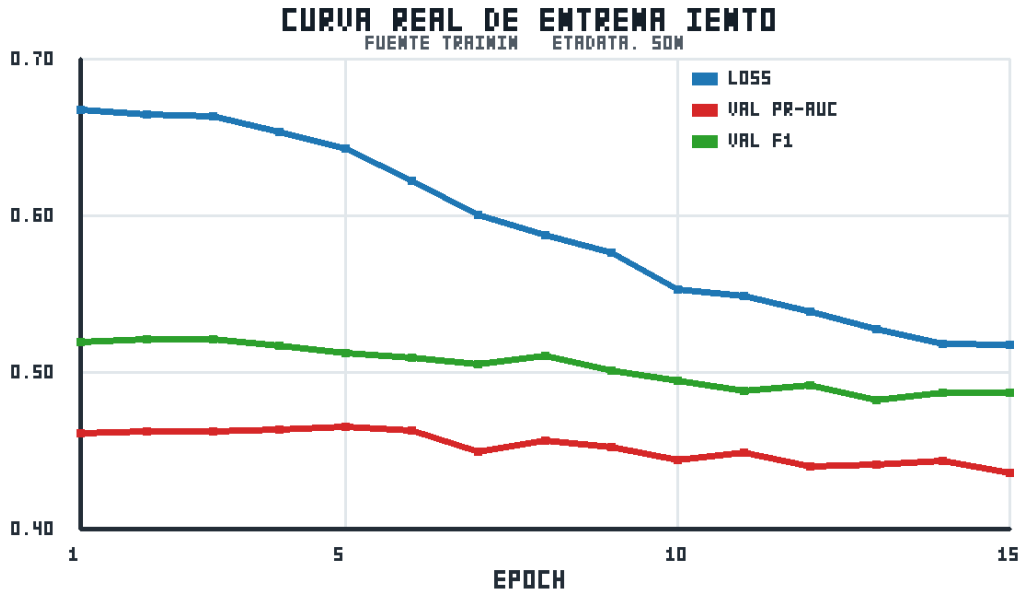
La accuracy debe interpretarse junto con el desbalance de clases: predecir siempre la clase negativa alcanzaría 92 %. En la variante calibrada, el recall de 0,4875 significa que se identificaron 117 de 240 positivos sintéticos y se omitieron 123. Por este motivo se priorizan PR-AUC, F1 y el análisis de falsos negativos, especialmente porque el objetivo del sistema es evitar que perfiles potencialmente valiosos pasen inadvertidos.

6.2.2 Comparación con baseline

La comparación incluye PlayerNet crudo, PlayerNet calibrado, regresión logística balanceada y un promedio simple de atributos. La regresión logística supera levemente a PlayerNet calibrado en ROC-AUC, PR-AUC y F1. Frente a PlayerNet crudo, obtiene ROC-AUC y F1 apenas superiores, mientras que PlayerNet crudo alcanza el mayor PR-AUC. Las diferencias son pequeñas y no respaldan una superioridad global de la red neuronal.

Modelo	Accuracy	ROC-AUC	PR-AUC	F1
PlayerNet sigmoid crudo	0,9300	0,9203	0,5461	0,5291
PlayerNet calibrado isotónico	0,9303	0,9174	0,5241	0,5282
LogisticRegression balanced	0,9310	0,9205	0,5378	0,5327
Promedio simple de atributos	0,8960	0,8390	0,3513	0,4201

TABLA 6-4. COMPARACIÓN CON BASELINES.



Como se observa en la Figura 6-1, la curva se construye con la historia real persistida: training loss, PR-AUC de validación y F1 de validación por época. La corrida no registró validation loss, por lo que esa serie no se incorpora ni se estima de manera retrospectiva.

FIGURA 6-1. CURVA REAL DE ENTRENAMIENTO GENERADA DESDE TRAINING_METADATA.JSON.

6.2.3 Evidencia operativa del deploy

Como evidencia operativa adicional, la Tabla 6-5 presenta el smoke HTTP ejecutado contra Render el 20/05/2026. El primer request refleja el arranque frío del plan Free; las rutas principales respondieron correctamente en esa verificación. Un control posterior del 10/07/2026 terminó por timeout, por lo que no se afirma disponibilidad continua actual.

Ruta	Método	Status	Tiempo	Resultado
/health	GET	200	109.36 s	OK; arranque frío Render Free
/login	GET	200	0.39 s	Pantalla de login renderizada

Ruta	Método	Status	Tiempo	Resultado
/login	POST	200	4.96 s	Login admin OK; redirección final a /players
/dashboard	GET	200	7.86 s	Panel general renderizado
/players	GET	200	0.38 s	Listado de jugadores
/compare	GET	200	0.43 s	Comparador 1v1
/compare/multi	GET	200	7.73 s	Comparador múltiple
/settings	GET	200	0.44 s	Configuración protegida
/players/import	GET	200	0.39 s	Carga masiva

TABLA 6-5. SMOKE REAL DEL DEPLOY EN RENDER.

Las siguientes figuras fueron capturadas desde el despliegue público en Render mediante una sesión autenticada. Cada captura documenta un flujo funcional del MVP y se interpreta como evidencia fechada, no como garantía de disponibilidad permanente.

La Figura 6-2 muestra la pantalla de acceso y la diferenciación de roles disponible en el MVP.

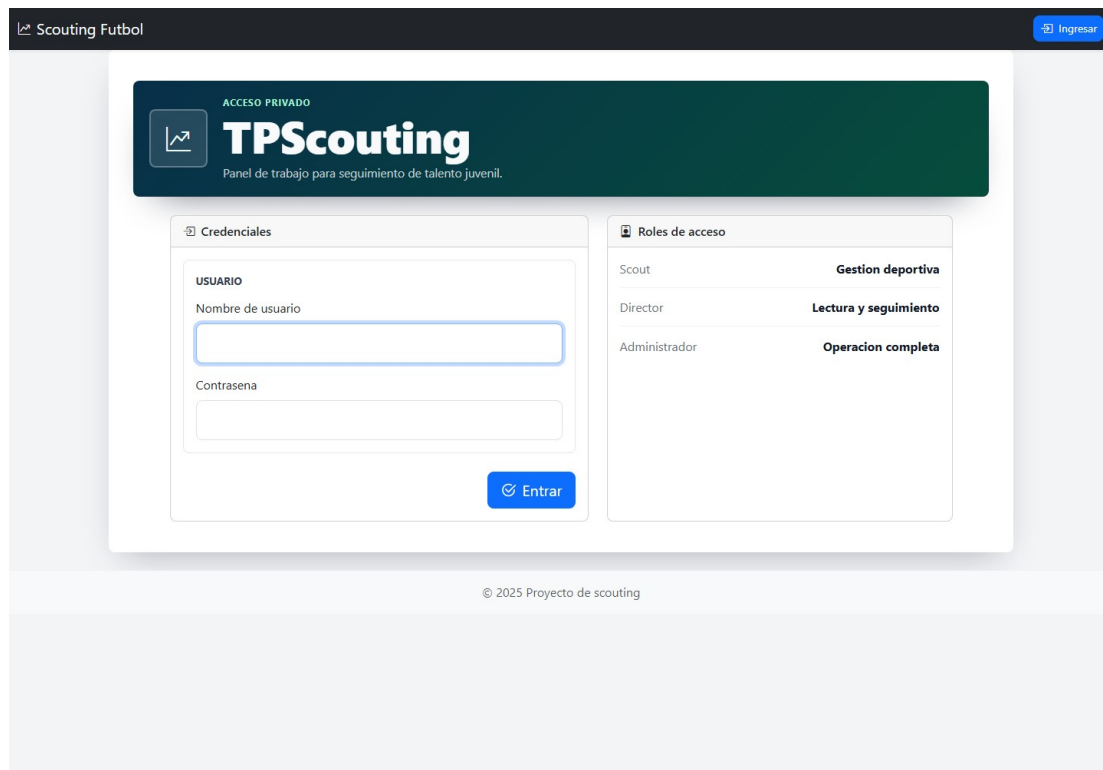
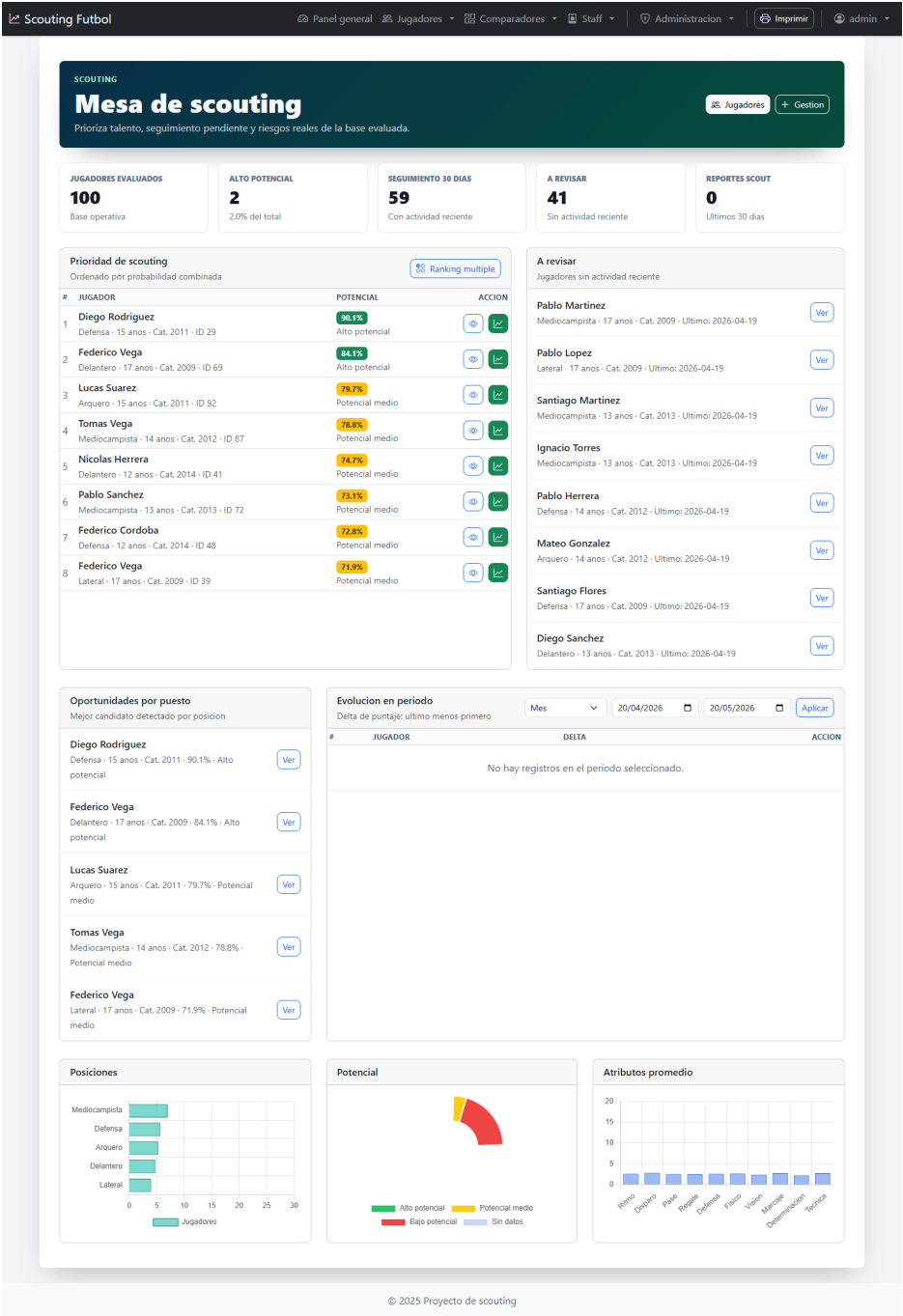


FIGURA 6-2. LOGIN DEL MVP DESPLEGADO EN RENDER.

La Figura 6-3 presenta el panel general, que concentra indicadores, prioridades y visualizaciones para apoyar el seguimiento.



Oportunidades por puesto
Mejor candidato detectado por posicion

Diego Rodriguez
Defensa · 15 años · Cat. 2011 · 90.1% · Alto potencial [Ver](#)

Federico Vega
Delantero · 17 años · Cat. 2009 · 84.1% · Alto potencial [Ver](#)

Lucas Suarez
Arquero · 15 años · Cat. 2011 · 79.7% · Potencial medio [Ver](#)

Tomas Vega
Mediocampista · 14 años · Cat. 2012 · 78.8% · Potencial medio [Ver](#)

Federico Vega
Lateral · 17 años · Cat. 2009 · 71.9% · Potencial medio [Ver](#)

Evolucion en periodo
Delta de puntaje: ultimo menos primero

Mes [Aplicar](#)

#	JUGADOR	DELTA	ACCION
No hay registros en el periodo seleccionado.			

Posiciones

Posición	Jugadores
Mediocampista	10
Defensa	5
Arquero	5
Delantero	5
Lateral	5

Potencial

Potencial	Cantidad	Porcentaje
Alto potencial	2	2.0%
Potencial medio	57	57.0%
Bajo potencial	41	41.0%
Sin datos	0	0.0%

Atributos promedio

Atributo	Promedio
Puntaje	5
Defensa	5
Pasa	5
Finesa	5
Velocidad	5
Fuerza	5
Resistencia	5
Mentalidad	5
Determinación	5
Técnica	5

FIGURA 6-3. PANEL GENERAL O MESA DE SCOUTING CON MÉTRICAS ACCIONABLES.

La Figura 6-4 muestra el listado paginado y filtrable de jugadores con edad, categoría y banda de potencial.

Scouting Futbol

Panel generalJugadoresComparadoresStaffAdministraciónImprimiradmin

Jugadores en seguimiento

Resultados: 100Página 1 de 5Panel generalGestionar

Organiza el seguimiento de talento juvenil y accede a cada ficha con un clic.

Buscar por nombreEj: Juan Perez

PosicionTodas

ClubTodos

PaisTodos

Ordenar por atributoSin orden especifico

☐ Solo alto potencial

Filtrar

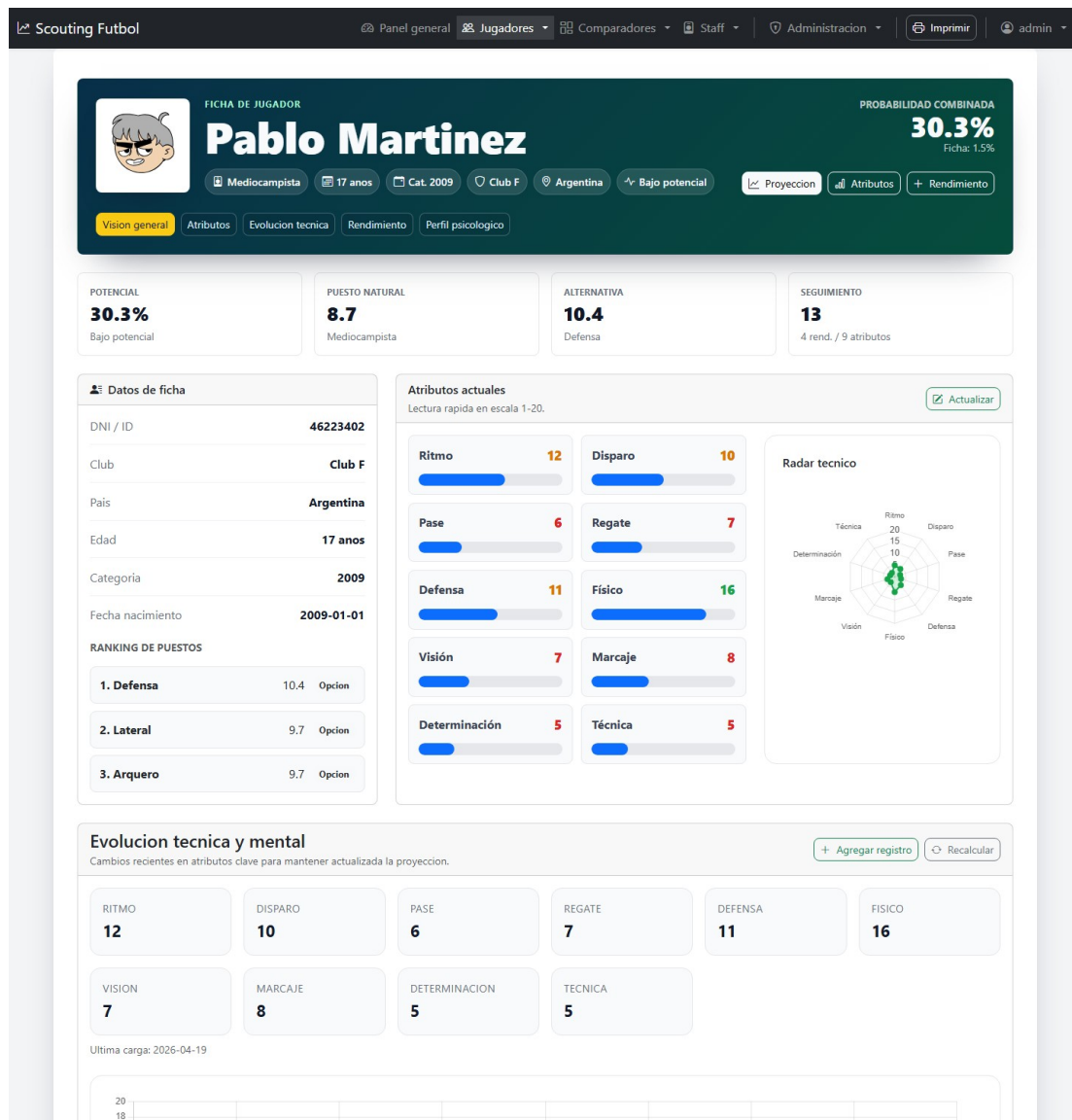
Nombre	Posición	Edad	Categoría	Alternativa	Club	País	Potencial / Prob.	Acciones
 Pablo Martinez DNI 46223402	Mediocampista	17 años	Cat. 2009	Defensa Rendimiento en posición: 6.65	Club F	Argentina	Bajo potencial 30.3%	Ver detalle Atributos Proyeccion
 Pablo Lopez DNI 25940381	Lateral	17 años	Cat. 2009	Defensa Rendimiento en posición: 11.34	Club C	Paraguay	Bajo potencial 55.8%	Ver detalle Atributos Proyeccion
 Tomas Vega DNI 23401487	Mediocampista	14 años	Cat. 2012	Arquero Rendimiento en posición: 13.34	Club D	Paraguay	Potencial medio 78.8%	Ver detalle Atributos Proyeccion
 Hernan Gonzalez DNI 19888400	Lateral	18 años	Cat. 2008	Delantero Rendimiento en posición: 6.56	Club A	Chile	Bajo potencial 31.7%	Ver detalle Atributos Proyeccion
 Santiago Martinez DNI 53618123	Mediocampista	13 años	Cat. 2013	Mediocampista Rendimiento en posición: 11.25	Club F	Argentina	Potencial medio 70.7%	Ver detalle Atributos Proyeccion
 Mateo Garcia DNI 58051262	Lateral	17 años	Cat. 2009	Defensa Rendimiento en posición: 10.13	Club D	Brasil	Bajo potencial 33.8%	Ver detalle Atributos Proyeccion
 Ignacio Torres DNI 33062477	Mediocampista	13 años	Cat. 2013	Delantero Rendimiento en posición: 8.7	Club E	Argentina	Bajo potencial 32.5%	Ver detalle Atributos Proyeccion
 Hernan Ferreira DNI 15354748	Arquero	18 años	Cat. 2008	Defensa Rendimiento en posición: 12.45	Escuela de Futbol	Paraguay	Potencial medio 66.6%	Ver detalle Atributos Proyeccion
 Pablo Herrera DNI 24304543	Defensa	14 años	Cat. 2012	Delantero Rendimiento en posición: 10.36	Club F	Brasil	Bajo potencial 53.3%	Ver detalle Atributos Proyeccion
 Pablo Ferreira DNI 54519263	Lateral	13 años	Cat. 2013	Defensa Rendimiento en posición: 13.63	Club B	Brasil	Bajo potencial 54.6%	Ver detalle Atributos Proyeccion
 Mateo Gonzalez DNI 52566108	Arquero	14 años	Cat. 2012	Delantero Rendimiento en posición: 10.45	Club B	Paraguay	Bajo potencial 52.6%	Ver detalle Atributos Proyeccion
 Santiago Ferreira DNI 34777665	Arquero	12 años	Cat. 2014	Defensa Rendimiento en posición: 10.75	Club F	Brasil	Bajo potencial 43.2%	Ver detalle Atributos Proyeccion
 Marcelo Bautista DNI 25887173	Mediocampista	13 años	Cat. 2013	Delantero Rendimiento en posición: 7.64	Club C	Argentina	Bajo potencial 29.5%	Ver detalle Atributos Proyeccion
 Gabriel Perez DNI 4090427	Arquero	15 años	Cat. 2011	Arquero Rendimiento en posición: 9.7	Club C	Uruguay	Bajo potencial 33.6%	Ver detalle Atributos Proyeccion
 Gabriel Cordoba DNI 57565775	Arquero	14 años	Cat. 2012	Defensa Rendimiento en posición: 14.35	Club B	Paraguay	Potencial medio 62.7%	Ver detalle Atributos Proyeccion
 Santiago Garcia DNI 22874170	Delantero	16 años	Cat. 2010	Lateral Rendimiento en posición: 9.6	Club D	Uruguay	Bajo potencial 37.1%	Ver detalle Atributos Proyeccion
 Santiago Flores DNI 55340576	Defensa	17 años	Cat. 2009	Arquero Rendimiento en posición: 12.11	Club E	Chile	Potencial medio 69.0%	Ver detalle Atributos Proyeccion
 Diego Alvarez DNI 17332920	Mediocampista	18 años	Cat. 2008	Delantero Rendimiento en posición: 10.16	Club D	Argentina	Bajo potencial 36.4%	Ver detalle Atributos Proyeccion
 Diego Sanchez DNI 16258758	Delantero	13 años	Cat. 2013	Mediocampista Rendimiento en posición: 8.77	Escuela de Futbol	Brasil	Bajo potencial 29.9%	Ver detalle Atributos Proyeccion
 Diego Perez DNI 59023381	Mediocampista	15 años	Cat. 2011	Arquero Rendimiento en posición: 8.78	Escuela de Futbol	Brasil	Bajo potencial 31.4%	Ver detalle Atributos Proyeccion

Página 1 de 5

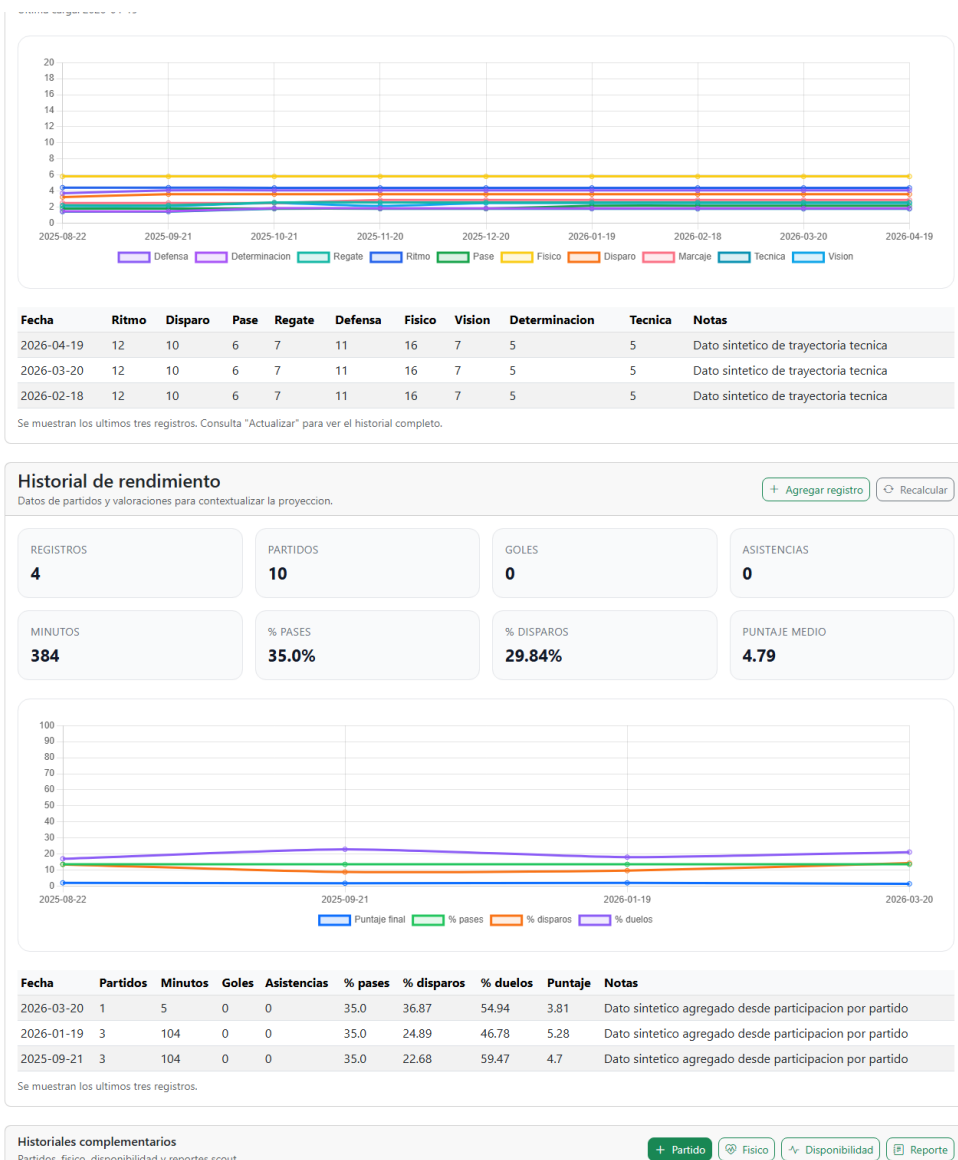
© 2025 Proyecto de scouting

FIGURA 6-4. LISTADO PAGINADO DE JUGADORES CON EDAD, CATEGORÍA Y POTENCIAL.

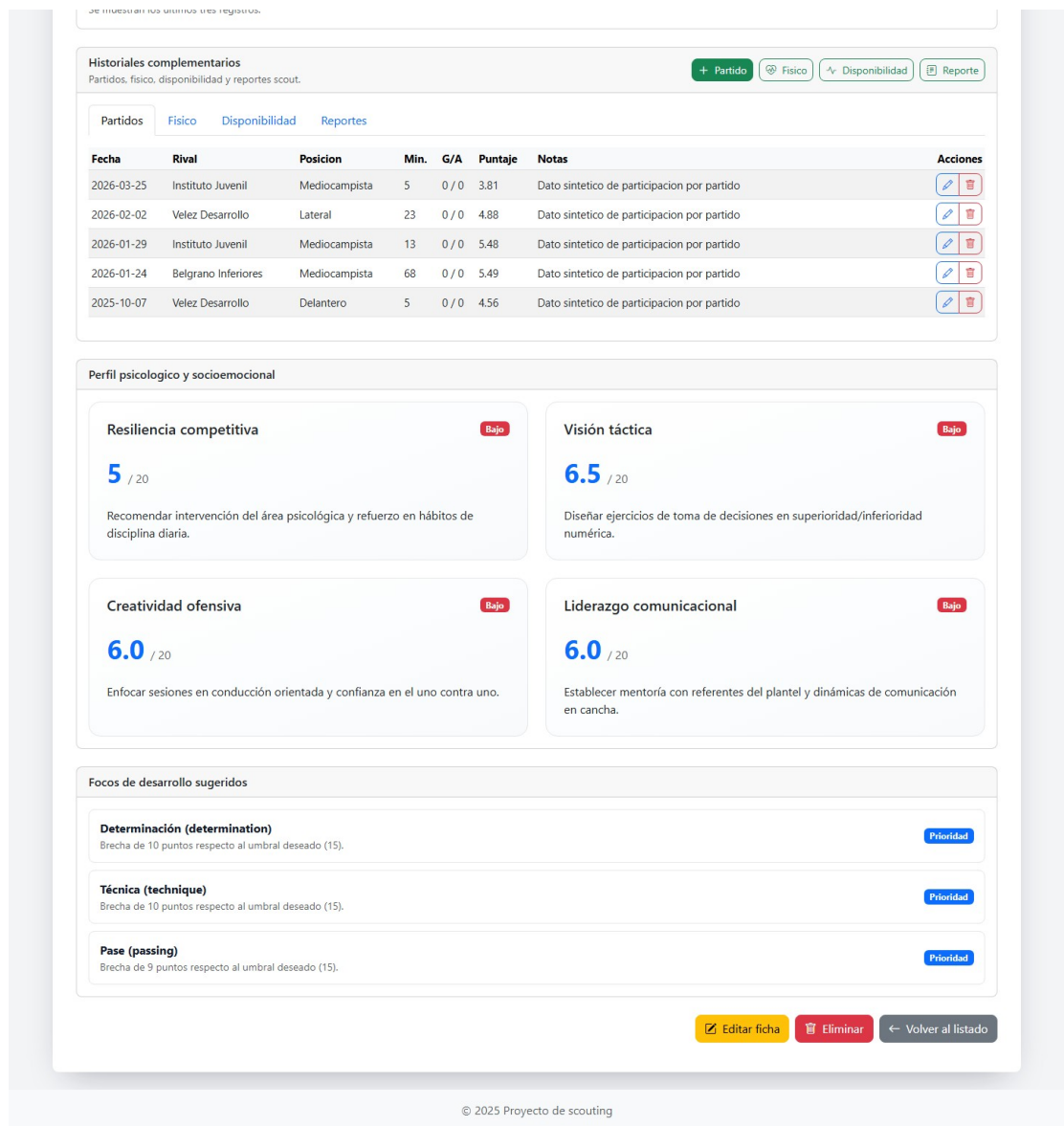
La Figura 6-5 reúne la ficha integral del jugador, sus atributos, historiales y acciones de mantenimiento.



a) Perfil, atributos y resumen de rendimiento.



b) Historiales, gráficos y evaluaciones periódicas.



c) Reportes, disponibilidad y acciones de mantenimiento.

FIGURA 6-5. FICHA DE JUGADOR CON HISTORIAL, ATRIBUTOS Y ACCIONES CRUD EN MODALES.

La Figura 6-6 presenta la vista de predicción, donde se exponen señales de decisión y evolución histórica.

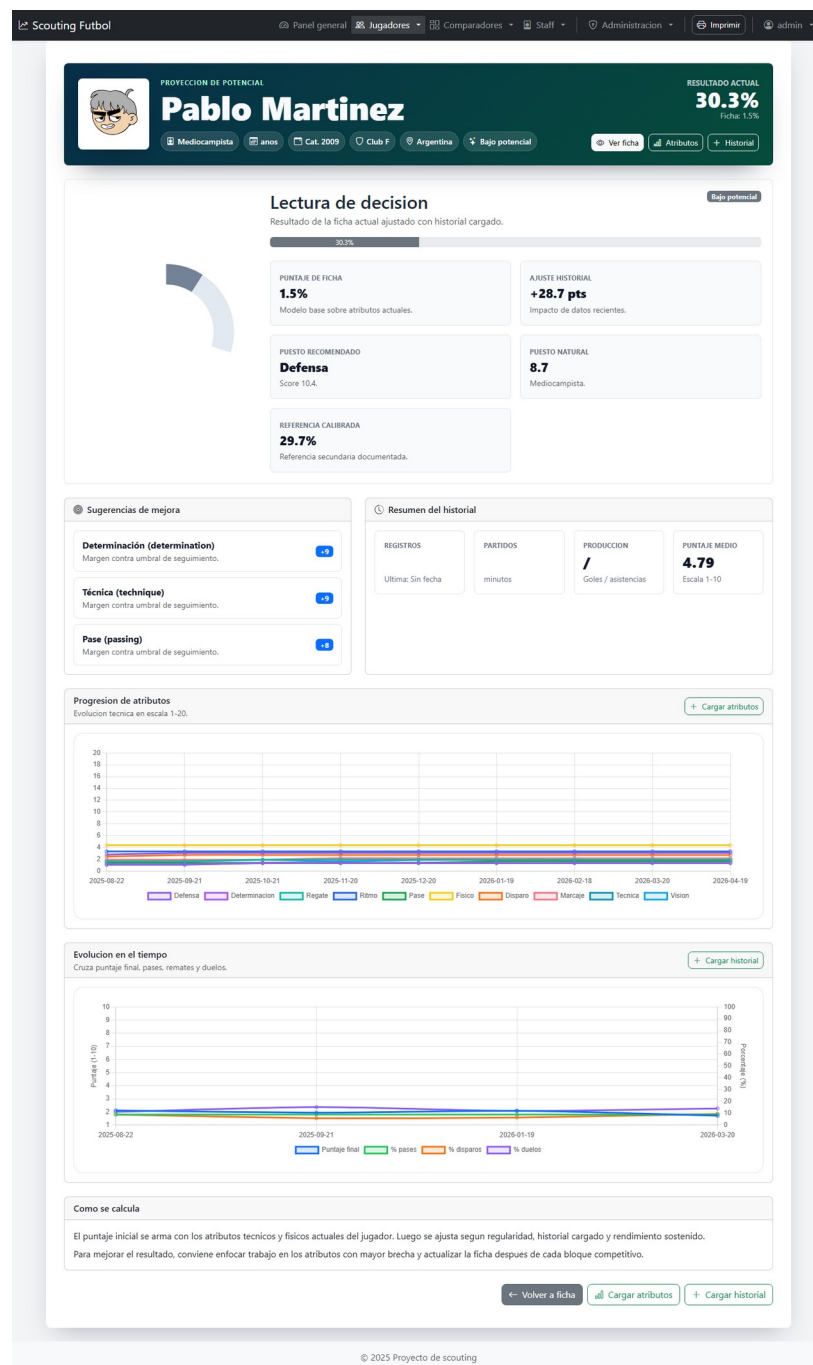


FIGURA 6-6. VISTA DE PREDICCIÓN DE POTENCIAL DEL JUGADOR.

La Figura 6-7 muestra el comparador múltiple utilizado para analizar perfiles en forma simultánea.

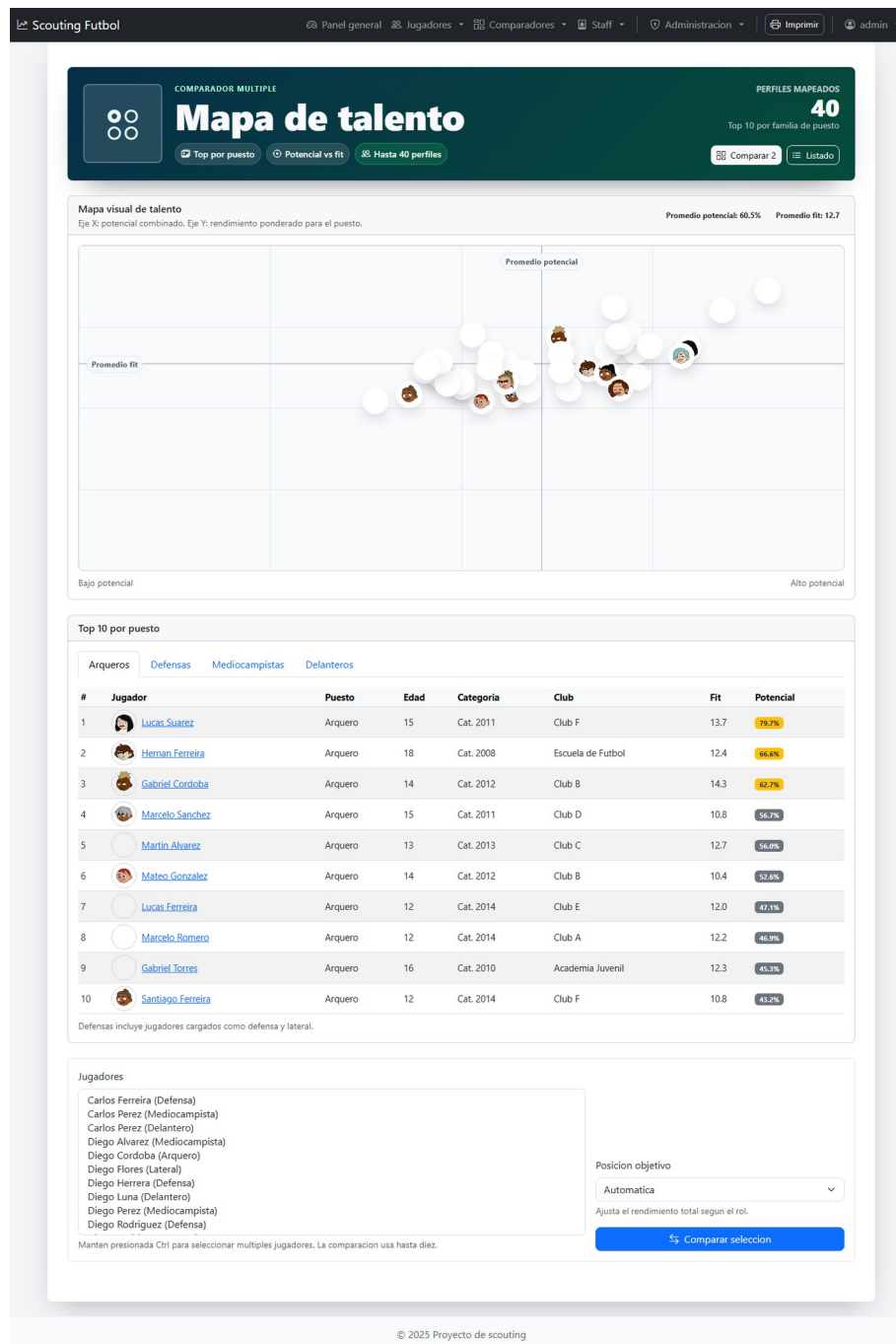


FIGURA 6-7. COMPARADOR MÚLTIPLE DE JUGADORES.

6.2.4 Evidencia de integración continua

La ejecución pública CI #71 finalizó con estado success para el commit fa8a50d1173f2760329a45f11bf93a709721235. Los jobs de Python 3.11 y 3.12 completaron instalación, tests y carga de artefactos de cobertura. Esta evidencia no

certifica el HEAD local 7d7680e, que durante la auditoría se encontraba un commit por delante.

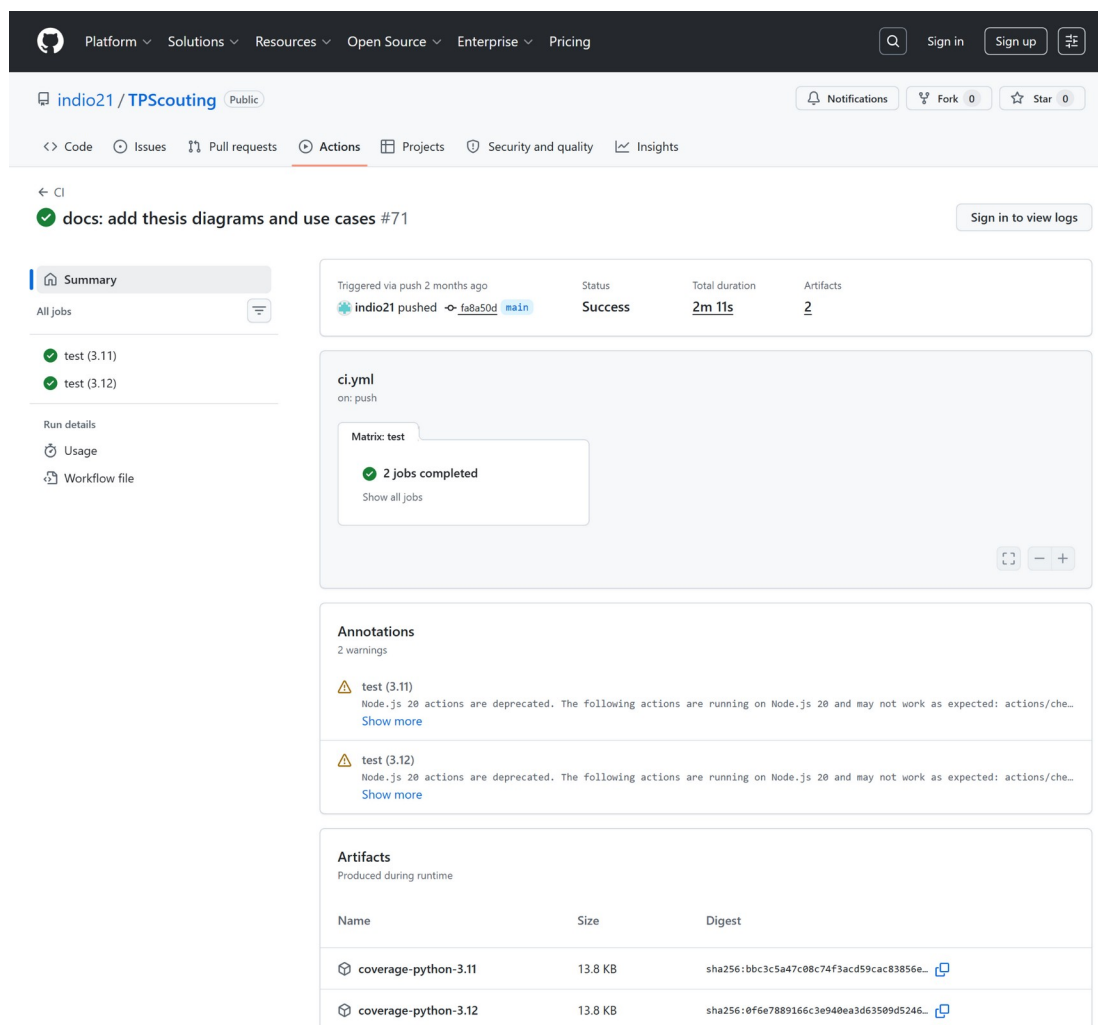


FIGURA 6-8. RESUMEN PÚBLICO DE LA EJECUCIÓN CI #71 EN GITHUB ACTIONS.

6.3 Limitaciones observadas en la validación

- Los datos y los outcomes son sintéticos; las métricas no representan desempeño real en un club.
- El target temporal utiliza cuantiles y cuotas calculados sobre el conjunto completo antes del split, por lo que validation y test no son totalmente independientes en la definición de sus etiquetas.
- Las métricas evalúan PlayerNet crudo y calibrado, pero no el score combinado mostrado por la interfaz.

- La corrida no registra validation loss, duración total ni SHA Git exacto.
- La clase positiva representa cerca del 8 %; el recall calibrado de 0,4875 implica una proporción relevante de falsos negativos sintéticos.
- Render Free puede presentar arranque frío, timeout o indisponibilidad temporal; la evidencia operativa debe citarse con fecha.
- La caché, el rate limiting y los locks son locales al proceso; una operación productiva multi-instancia requeriría servicios distribuidos.

7. CONCLUSIÓN

7.1 Conclusiones

El desarrollo demuestra la factibilidad de integrar una aplicación web de scouting juvenil con persistencia, historiales, visualización, comparadores y un módulo de aprendizaje automático. El resultado es adecuado como MVP académico y evidencia un flujo técnico reproducible; no constituye una solución productiva ni una validación deportiva externa.

El principal aporte no consiste en reemplazar la observación humana, sino en estructurar información dispersa, hacer trazables los criterios y ofrecer una base consistente para comparar jugadores. Los experimentos muestran además que la complejidad del modelo no garantiza superioridad: la regresión logística y PlayerNet alcanzaron resultados similares, con ventajas diferentes según la métrica, y el score combinado de la aplicación todavía requiere una evaluación independiente.

7.2 Cumplimiento de objetivos

Objetivo declarado en 1.2	Estado	Evidencia y alcance
Desarrollar una aplicación con IA y análisis de datos para apoyar evaluación y proyección	Cumplido a nivel de MVP	Aplicación funcional, PlayerNet integrado, paneles, comparadores, pruebas y despliegue documentado; falta validación deportiva externa.
Optimizar la evaluación y reducir subjetividad	Cumplido parcialmente	Se estructuraron escalas, historiales, reglas y modelos; no se comparó el proceso contra scouts ni se midió reducción de subjetividad.
Identificar talentos subrepresentados	No validado	La funcionalidad de predicción y filtrado existe, pero no se evaluó con jugadores reales ni contra un proceso tradicional.
Mejorar la toma de decisiones	Cumplido a nivel funcional	El panel, la ficha y los comparadores aportan información; no se realizó un estudio de uso con entrenadores o scouts.
Promover la igualdad de oportunidades	Contribución potencial	El stack y el MVP reducen barreras de acceso, pero no se midió adopción ni impacto social en clubes con menos recursos.
Optimizar la gestión de recursos	Cumplido parcialmente	El sistema permite priorizar y organizar información; no se cuantificaron costos, tiempos ni eficiencia antes y después.

TABLA 7-1. CUMPLIMIENTO DE OBJETIVOS.

7.3 Limitaciones

- Dataset y outcomes completamente sintéticos, sin validación longitudinal con jugadores reales.
- Construcción del target con cuantiles y cuotas globales antes de la partición, lo que limita la independencia de validation y test.
- Ausencia de evaluación independiente del score combinado utilizado por la interfaz.
- Sin validation loss, duración total ni vinculación criptográfica de la corrida con un commit exacto.
- Sin integración de video, tracking, GPS o proveedores comerciales.
- Persistencia y operación suficientes para una demo, pero no para uso multiusuario intensivo o multi-instancia.
- PlayerNet no presenta una superioridad global frente a la regresión logística en la corrida actual.

7.4 Lecciones aprendidas

El proceso mostró que una arquitectura sencilla, trazable y verificable fue más adecuada para el alcance que una solución distribuida. La comparación experimental confirmó que una red neuronal más compleja no garantiza una mejora frente a un modelo lineal bien configurado.

También se comprobó que la definición del target, la separación temporal de los datos y la trazabilidad de los artefactos son tan importantes como la arquitectura del modelo. Para una siguiente iteración, la prioridad debería ser mejorar el diseño experimental y validar con datos reales antes de aumentar la complejidad algorítmica.

Por último, la integración entre desarrollo, pruebas, despliegue y documentación resultó esencial. Registrar métricas, hashes, cobertura, limitaciones y fechas de evidencia permite presentar resultados más honestos y reproducibles.

7.5 Trabajo futuro

- Validar el modelo con datos reales y resultados observables: minutos competitivos, promociones, consenso scout o firma profesional.

- Incorporar auditoría de cambios, logs estructurados y cache/rate limiting distribuido.
- Separar más lógica de app.py hacia servicios o una app factory si el proyecto crece.
- Agregar exportaciones formales, reportes PDF y flujos de aprobación por rol.
- Evaluar modelos alternativos y seleccionar el más adecuado según evidencia, no por complejidad técnica.

8. BIBLIOGRAFÍA

Las referencias externas se consolidan en esta sección. Los documentos internos del proyecto se trasladan a anexos para evitar confundir bibliografía con evidencia propia.

Sommerville, I. (2011). Ingeniería de software (9.^a ed.). Pearson Educación.

Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.^a ed.). McGraw-Hill.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.

PyTorch Foundation. (s. f.). PyTorch documentation. Recuperado el 15 de mayo de 2026 de <https://pytorch.org/docs/>

Pallets Projects. (s. f.). Flask documentation. Recuperado el 16 de mayo de 2026 de <https://flask.palletsprojects.com/>

SQLAlchemy. (s. f.). SQLAlchemy documentation. Recuperado el 17 de mayo de 2026 de <https://docs.sqlalchemy.org/>

OWASP Foundation. (s. f.). OWASP Top 10:2021. Recuperado el 18 de mayo de 2026 de <https://owasp.org/Top10/>

Hudl. (s. f.). Wyscout. Recuperado el 19 de mayo de 2026 de <https://www.hudl.com/products/wyscout>

FIFA. (s. f.). Talent Development. Recuperado el 20 de mayo de 2026 de <https://publications.fifa.com/es/talent-development/>

- van Arem, K. W., Goes-Smit, F., & Söhl, J. (2025). Forecasting the future development in quality and value of professional football players. *Applied Sciences*, 15(16), 8916. <https://doi.org/10.3390/app15168916>
- Lacan, S. (2024). Stacking-based deep neural network for player scouting in football. *arXiv*. <https://doi.org/10.48550/arXiv.2403.08835>
- Decroos, T., Bransen, L., Van Haaren, J., & Davis, J. (2019). Actions speak louder than goals: Valuing player actions in soccer. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1851-1861. <https://doi.org/10.1145/3292500.3330758>
- Pappalardo, L., Cintia, P., Ferragina, P., Massucco, E., Pedreschi, D., & Giannotti, F. (2019). PlayeRank: Data-driven performance evaluation and player ranking in soccer via a machine learning approach. *ACM Transactions on Intelligent Systems and Technology*, 10(5), Article 59. <https://doi.org/10.1145/3343172>

9. ANEXOS TÉCNICOS DEL MVP

Esta sección reúne documentación propia del proyecto. No se presenta como bibliografía externa, sino como anexos técnicos verificables contra el código fuente actual.

9.1 Rutas principales

Archivo	Rutas relevantes
auth.py	/login, /logout, /register
players.py	/players, /player/<id>, /players/manage, /players/import, historiales y predicción
dashboard.py	/dashboard
compare.py	/compare, /compare/multi
staff.py	/coaches, /directors
settings.py	/settings
app.py	/, /health y registro de blueprints

TABLA 9-1. RUTAS PRINCIPALES.

9.2 Variables de entorno relevantes

Variable	Uso
APP_SECRET_KEY	Obligatoria en producción para firmar sesiones.
APP_DB_URL	Base operativa SQLite local o PostgreSQL en Render.
TRAINING_DB_URL	Base de entrenamiento cuando se ejecuta pipeline.
ADMIN_USERNAME / ADMIN_PASSWORD	Credenciales iniciales del administrador.
ALLOW_DEFAULT_ADMIN	Solo desarrollo local; no usar en producción.
CACHE_TTL_SECONDS / CACHE_MAX_ENTRIES	Cache in-memory configurable.
POTENTIAL_MEDIUM_THRESHOLD / POTENTIAL_HIGH_THRESHOLD	Umbrales documentados; default 0.60 y 0.80.
RENDER_SMOKE_BASE_URL	URL pública usada para smoke externo.

TABLA 9-2. VARIABLES DE ENTORNO RELEVANTES.

9.3 Glosario técnico

Término	Definición
MVP	Producto mínimo viable utilizado para validar el flujo principal del sistema.

Término	Definición
PlayerNet	Red neuronal propia del proyecto que recibe 68 variables y produce una probabilidad cruda de pertenencia a la clase positiva sintética.
Vector de características	Representación numérica de las variables de entrada utilizadas por el modelo.
ROC-AUC	Métrica de separación entre clases para clasificación binaria.
PR-AUC	Métrica útil con clase positiva minoritaria.
BCEWithLogitsLoss	Función de pérdida usada con logits en clasificación binaria.
CSRF	Ataque de falsificación de petición; mitigado con token en formularios POST.
Gunicorn	Servidor WSGI usado para ejecutar Flask en Render.
Render	Proveedor cloud utilizado para la demo pública.
SCRUM	Marco de trabajo ágil utilizado para organizar iteraciones, planificación y revisión del avance.
Sprint	Iteración de trabajo con objetivo acotado dentro de SCRUM.
Dashboard / panel general	Vista de control con indicadores, listas accionables y gráficos para apoyar decisiones.
SQLAlchemy	ORM de Python utilizado para mapear entidades del dominio a tablas relacionales.
PostgreSQL	Motor relacional administrado utilizado en el despliegue público de Render.
SQLite	Motor relacional liviano utilizado para desarrollo local y pruebas del MVP.
Logits	Salida cruda del modelo antes de convertirla a probabilidad mediante sigmoid.
Sigmoid	Función matemática que transforma logits en valores entre 0 y 1.
Baseline	Modelo o regla de referencia usado para comparar el desempeño de una propuesta.
Checkpoint	Archivo que guarda estado del modelo y metadata necesaria para cargarlo de forma compatible.
input_dim	Cantidad de features esperadas por el modelo al momento de inferir.
Smoke test	Prueba rápida de disponibilidad y rutas críticas del sistema desplegado.
CI	Integración continua; ejecución automatizada de pruebas en GitHub Actions.
pytest-cov	Extensión de pytest usada para medir cobertura de pruebas.
temporal_target_label	Variable objetivo binaria sintética utilizada en la corrida oficial; se construye con señales futuras simuladas y reglas temporales.

Término	Definición
Probabilidad cruda	Salida sigmoid directa de PlayerNet antes de calibración o combinación con reglas operativas.
Calibración isotónica	Reajuste no paramétrico de una probabilidad para mejorar su interpretación; no garantiza mejores métricas de ranking.
Score combinado	Valor operativo de la interfaz que integra probabilidad cruda, promedio histórico y ajuste posicional.
Modelo de referencia (baseline)	Modelo simple utilizado como punto de comparación; en este trabajo se usa regresión logística balanceada.

TABLA 9-3. GLOSARIO TÉCNICO.

9.4 Reproducción mínima

Los siguientes comandos corresponden al flujo documentado del repositorio. Deben ejecutarse desde la raíz del proyecto en Windows y con las variables de entorno necesarias. No se conserva el transcript exacto de la corrida original de entrenamiento.

Crear entorno virtual:

```
py -3.11 -m venv .venv
```

Instalar dependencias:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt -r requirements-dev.txt
```

Crear administrador:

```
.\.venv\Scripts\python.exe .\scouting_app\create_admin.py
```

Ejecutar la aplicación:

```
.\.venv\Scripts\python.exe .\scouting_app\app.py
```

Ejecutar pruebas y cobertura:

```
.\.venv\Scripts\python.exe -m pytest -q --cov=scouting_app --cov-report=term-missing
```

Generar datos de entrenamiento:

```
generate_data.py --num-players 20000 --db-url sqlite:///players_training.db --seed 42 --min-age 12 --max-age 18 --reset
```

Entrenar:

```
train_model.py ... --epochs 45 --lr 5e-4 --patience 10
```

Evaluar el modelo:

```
evaluate_saved_model.py --db-url sqlite:///players_training.db --metadata-path training_metadata.json
```

Smoke del despliegue:

```
.\.venv\Scripts\python.exe scripts\smoke_render.py --base-url <URL>
```

10. ANEXOS VISUALES Y EVIDENCIA

Los anexos visuales complementan la sección de pruebas con capturas del sistema, diagramas ampliados y evidencia de integración continua. Las evidencias se presentan con fecha y alcance para evitar interpretar una verificación histórica como disponibilidad permanente.

10.1 Evidencia Render

- URL pública validada históricamente el 20/05/2026: <https://tpscouting-mvp.onrender.com>.
- Login administrativo verificado en el smoke histórico del 20/05/2026.
- Rutas verificadas: /dashboard, /players, /compare, /compare/multi, /settings y /players/import.
- Healthcheck histórico documentado con 100 jugadores demo y faltantes críticos en 0; un control del 10/07/2026 finalizó por timeout.

10.2 Integración continua y documentos técnicos

- README.md: instalación, pruebas, deploy y limitaciones.
- RUNBOOK.md: operación, variables, smoke, Render y riesgos conocidos.
- docs/model_training_evidence.md: evidencia del entrenamiento y comparación de modelos.
- docs/auditoria_pendientes_2026-05-17.md: riesgos restantes y decisiones MVP.
- docs/cierre_pre_entrega_word_render_2026-05-18.md: cierre pre-entrega, deploy y smoke.

La Figura 10-9 conserva una vista ampliada de la ejecución CI #71, con jobs exitosos para Python 3.11 y 3.12 y dos artefactos de cobertura.

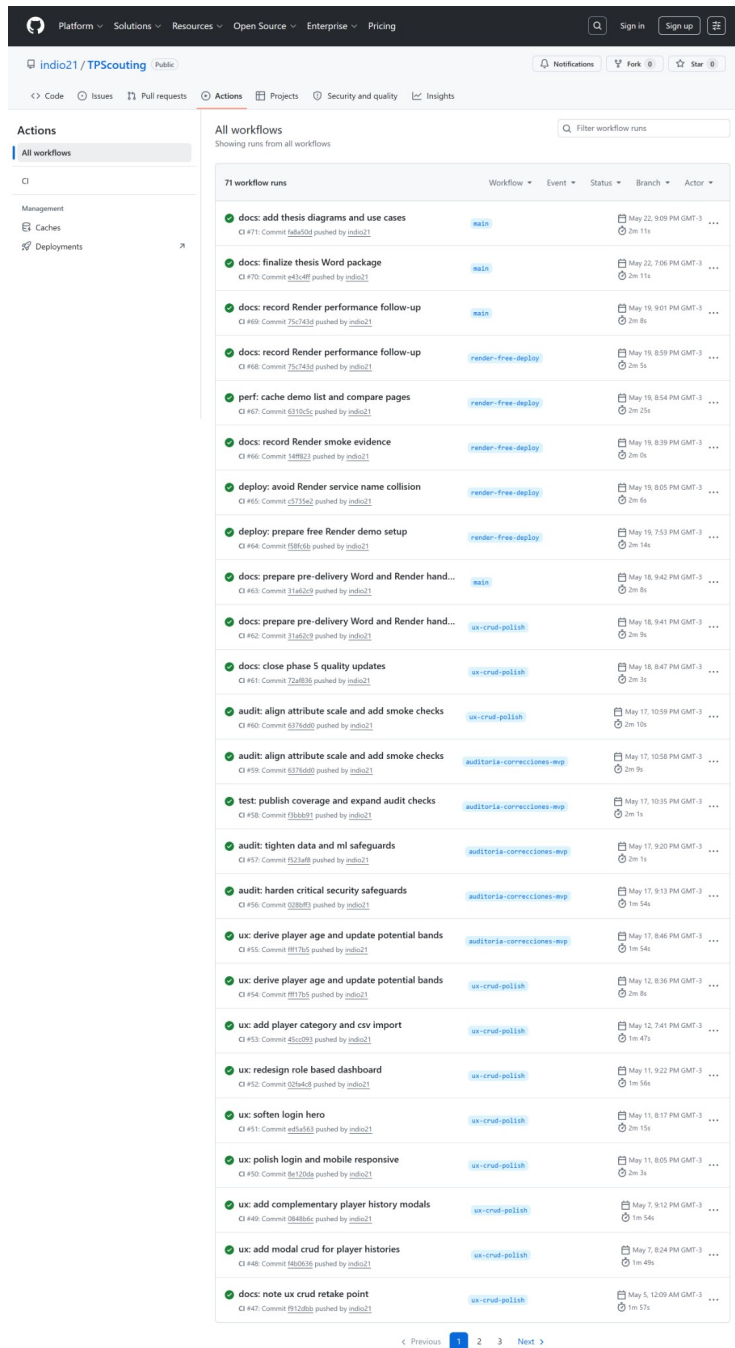


FIGURA 10-9. HISTORIAL PÚBLICO Y EJECUCIÓN EXITOSA DEL WORKFLOW CI DE TPSCOUTING.

10.3 Limitaciones documentadas del MVP

- Datos y outcomes sintéticos; las métricas no son extrapolables a rendimiento real sin validación externa.

- Render Free puede dormir el servicio y producir latencias iniciales elevadas.
- Cache, rate limiting y lock de pipeline son locales al proceso.
- La app está modularizada por blueprints, pero todavía no es una arquitectura de microservicios.
- La decisión de modelo debe revisarse con datos reales; el baseline logístico fue competitivo en la corrida actual.

10.4 Diagramas ampliados

Esta sección incorpora los diagramas técnicos en páginas independientes para facilitar su lectura. Las mismas figuras se mantienen previamente en los capítulos correspondientes como referencia contextual.

La Figura 10-1 presenta en tamaño ampliado los componentes del cliente, la aplicación Flask, servicios, persistencia y módulo de Machine Learning.

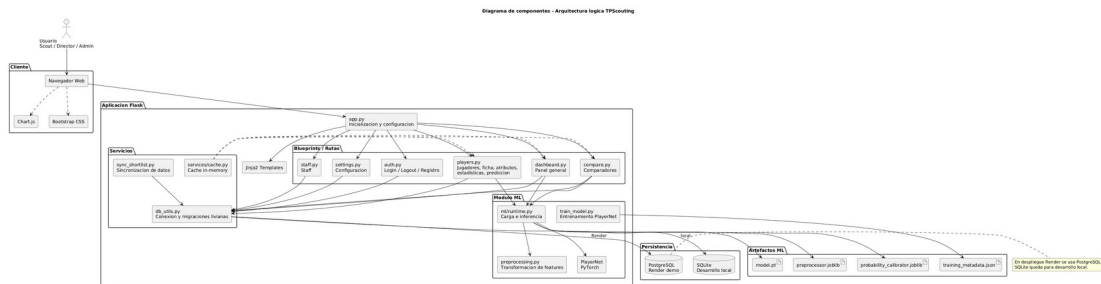


FIGURA 10-1. DIAGRAMA DE COMPONENTES DE TPSCOUTING AMPLIADO.

La Figura 10-2 presenta en tamaño ampliado las clases principales definidas en el modelo SQLAlchemy y sus relaciones.

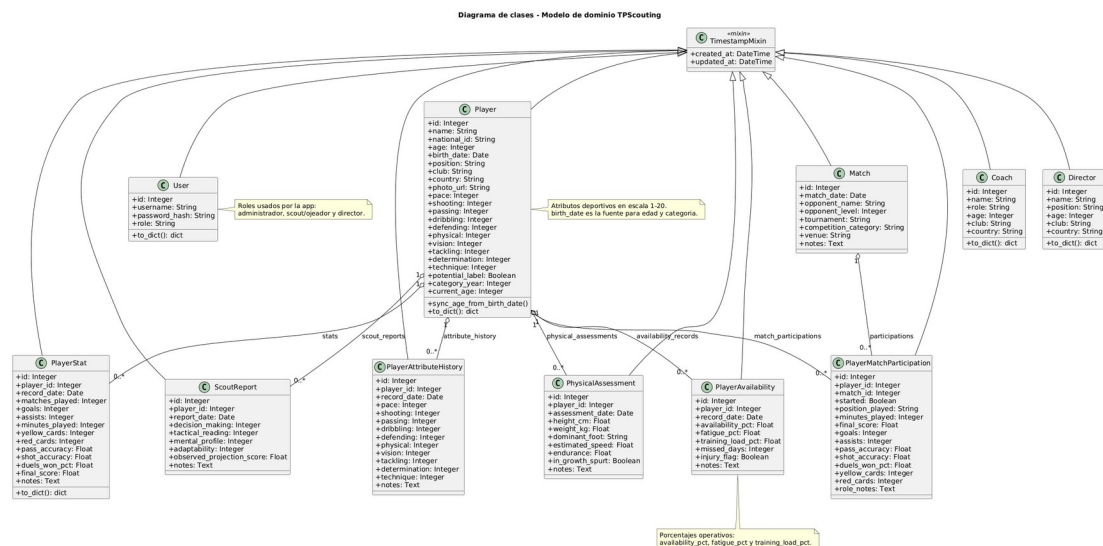


FIGURA 10-2. DIAGRAMA DE CLASES DEL MODELO DE DOMINIO AMPLIADO.

La Figura 10-3 presenta en tamaño ampliado el flujo de consulta de potencial de un jugador.

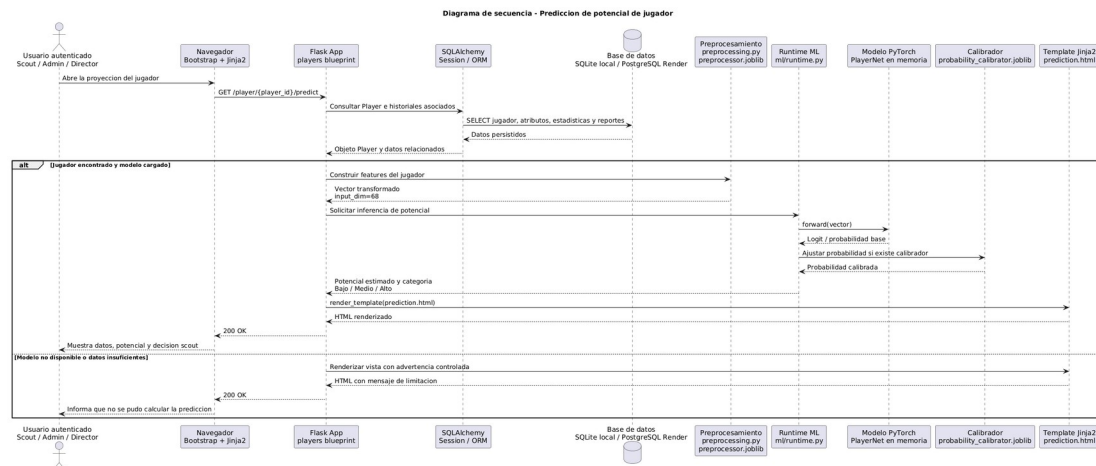


FIGURA 10-3. DIAGRAMA DE SECUENCIA DE PREDICCIÓN AMPLIADO.

La Figura 10-4 presenta en tamaño ampliado el flujo de carga del panel general, incluyendo cache y consultas a base de datos.

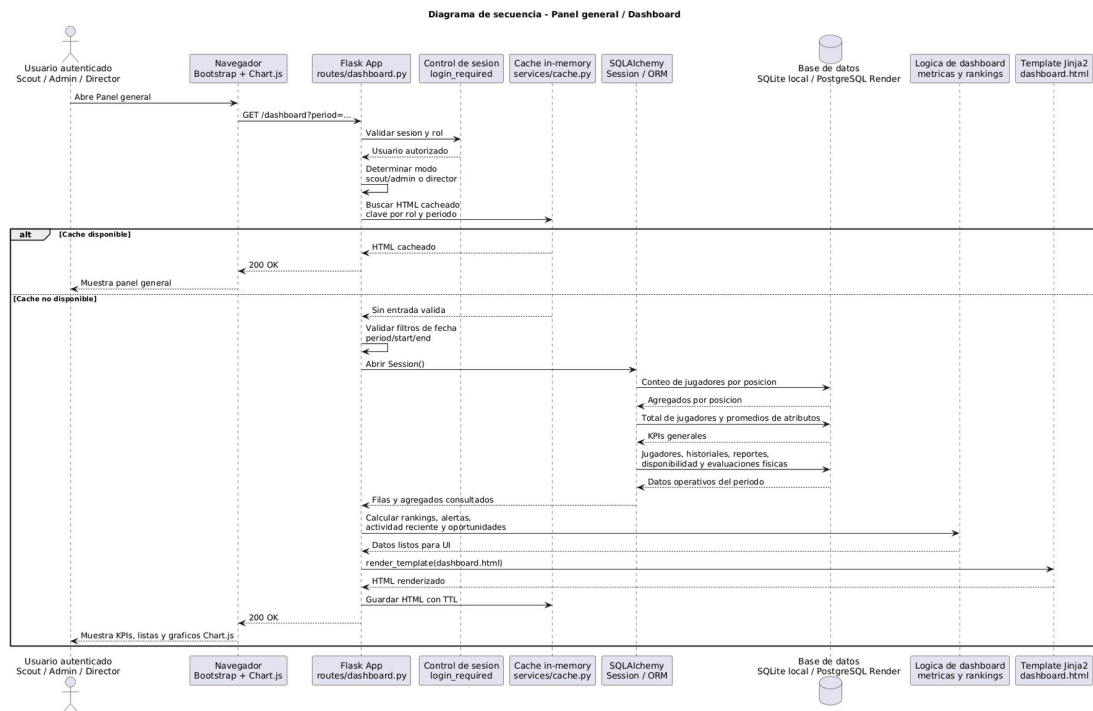


FIGURA 10-4. DIAGRAMA DE SECUENCIA DEL PANEL GENERAL AMPLIADO.

La Figura 10-5 presenta en tamaño ampliado la relación entre navegador, Render, Gunicorn, Flask, PostgreSQL y artefactos ML.

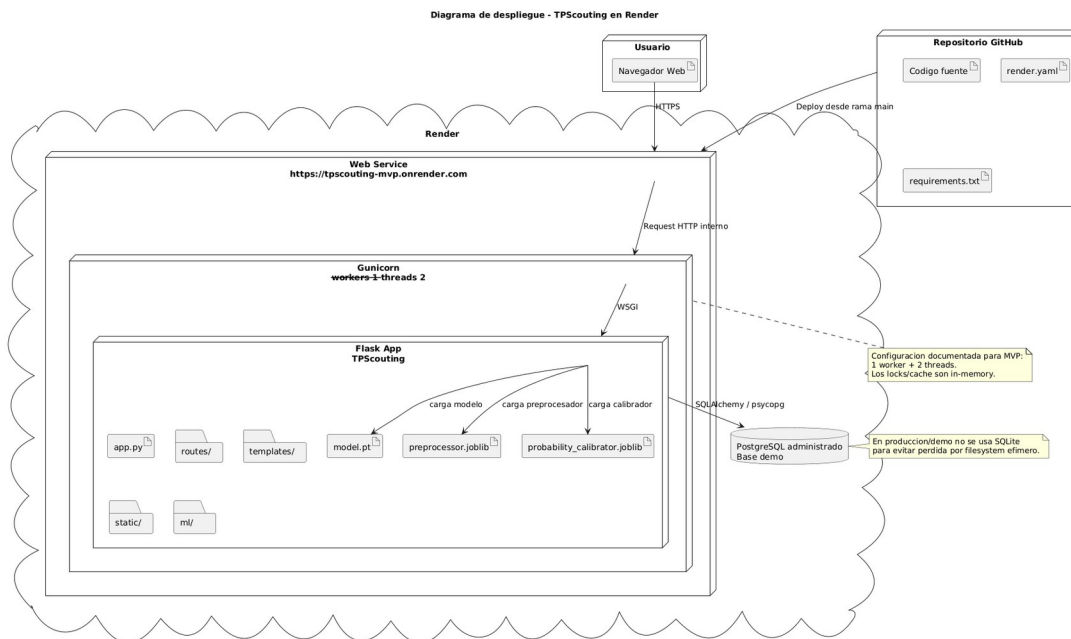


FIGURA 10-5. DIAGRAMA DE DESPLIEGUE EN RENDER AMPLIADO.

10.5 Casos de uso ampliados

Esta sección incorpora los diagramas de casos de uso en páginas independientes para facilitar su lectura.

La Figura 10-6 presenta en tamaño ampliado la vista general de casos de uso del sistema.

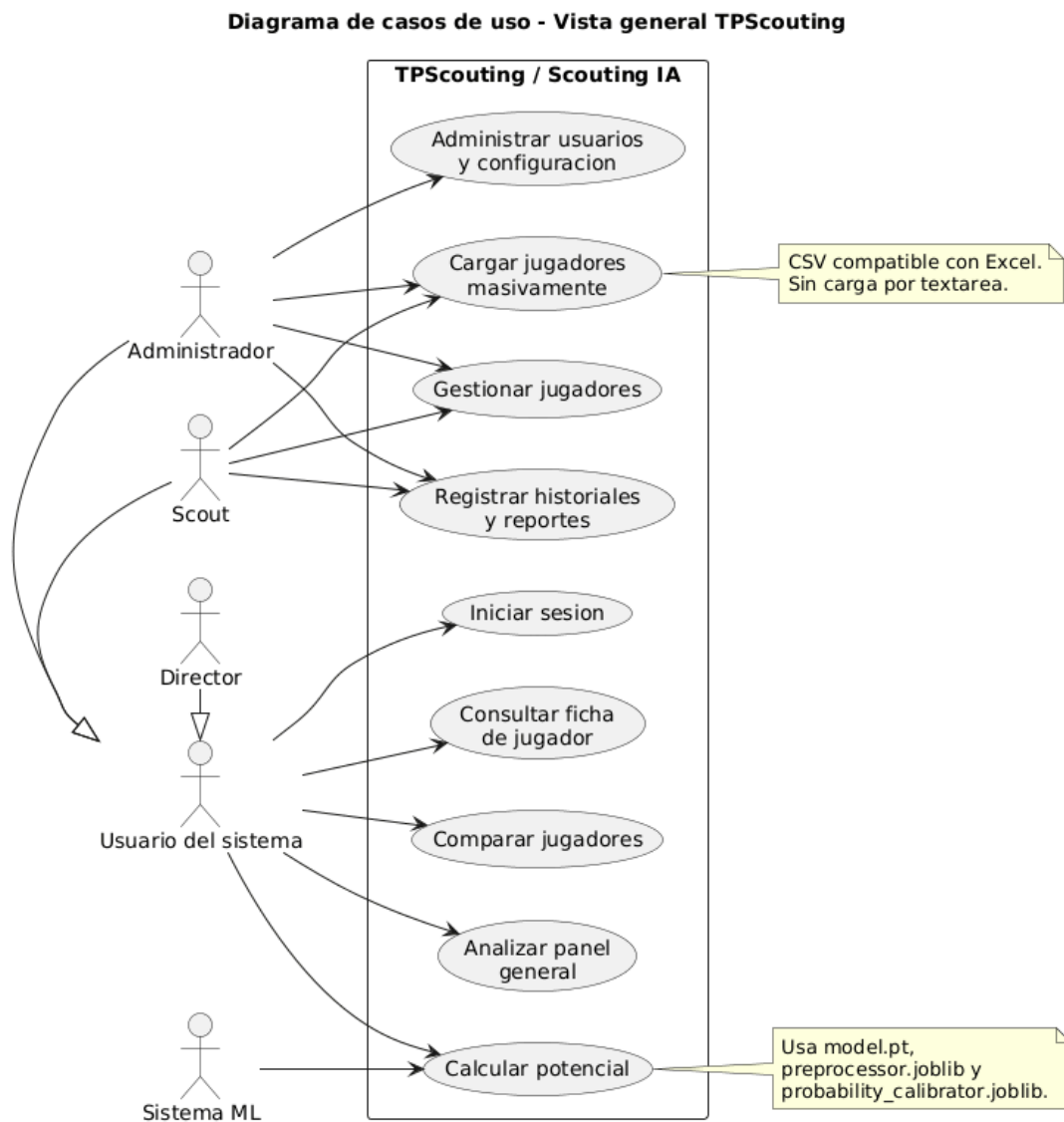


FIGURA 10-6. DIAGRAMA DE CASOS DE USO GENERAL AMPLIADO.

La Figura 10-7 presenta en tamaño ampliado los casos de uso del módulo de jugadores.

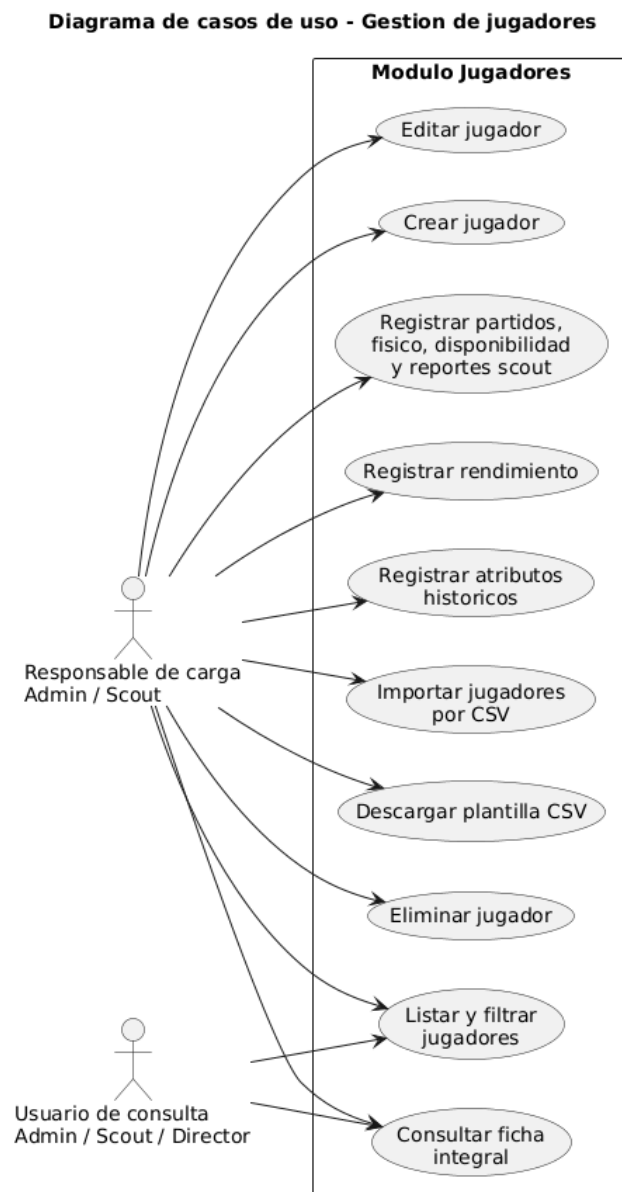


FIGURA 10-7. DIAGRAMA DE CASOS DE USO DE GESTIÓN DE JUGADORES AMPLIADO.

La Figura 10-8 presenta en tamaño ampliado los casos de uso de análisis, comparación y predicción.

Diagrama de casos de uso - Analisis y decision scout

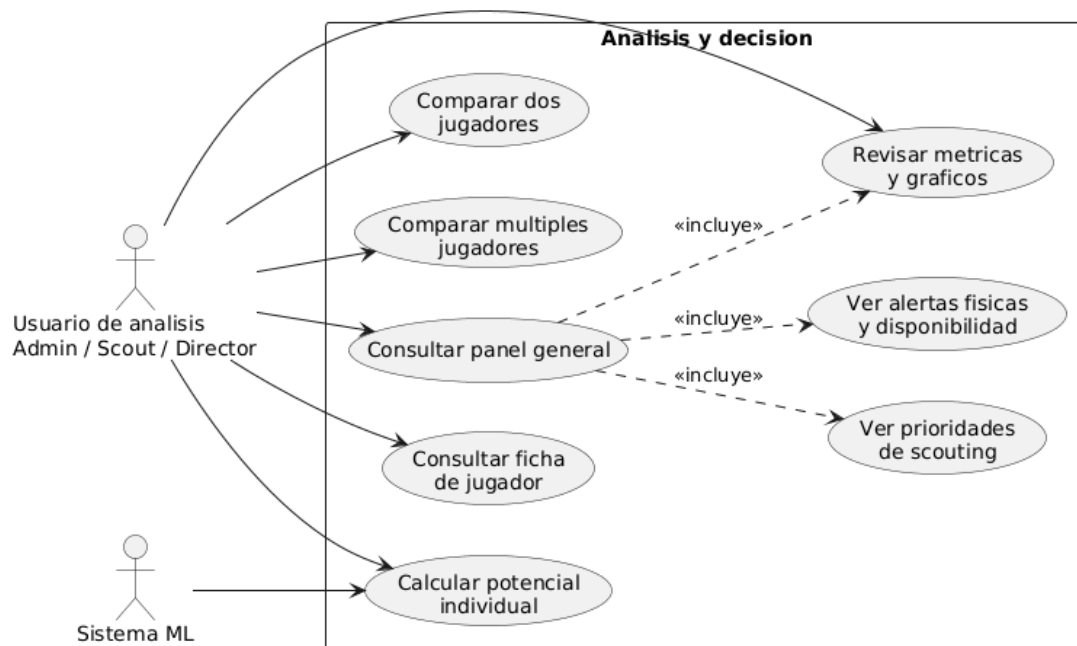


FIGURA 10-8. DIAGRAMA DE CASOS DE USO DE ANÁLISIS Y DECISIÓN SCOUT AMPLIADO.